Practical Assignment - 2

Name : Bhaliya Dhruvisha Ashokbhai

Roll No : 05

Subject : 304 Open Source Web Development

Course : MSc.ICT

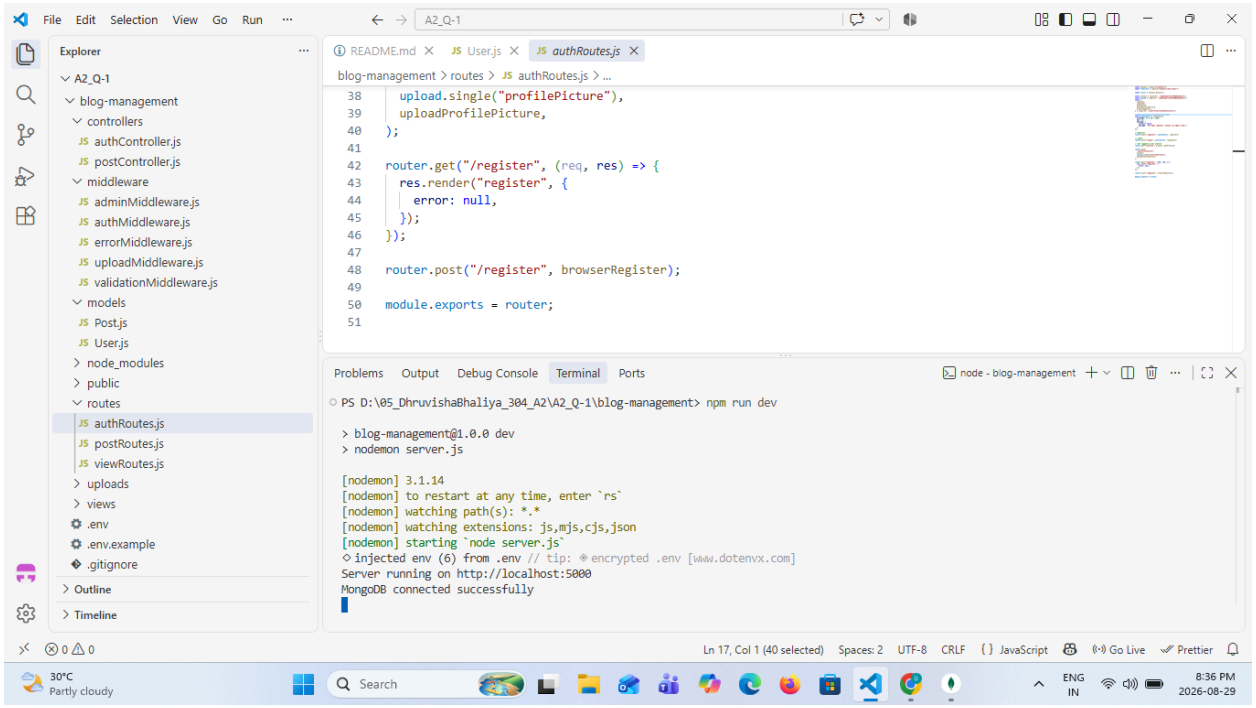
Semester : 3

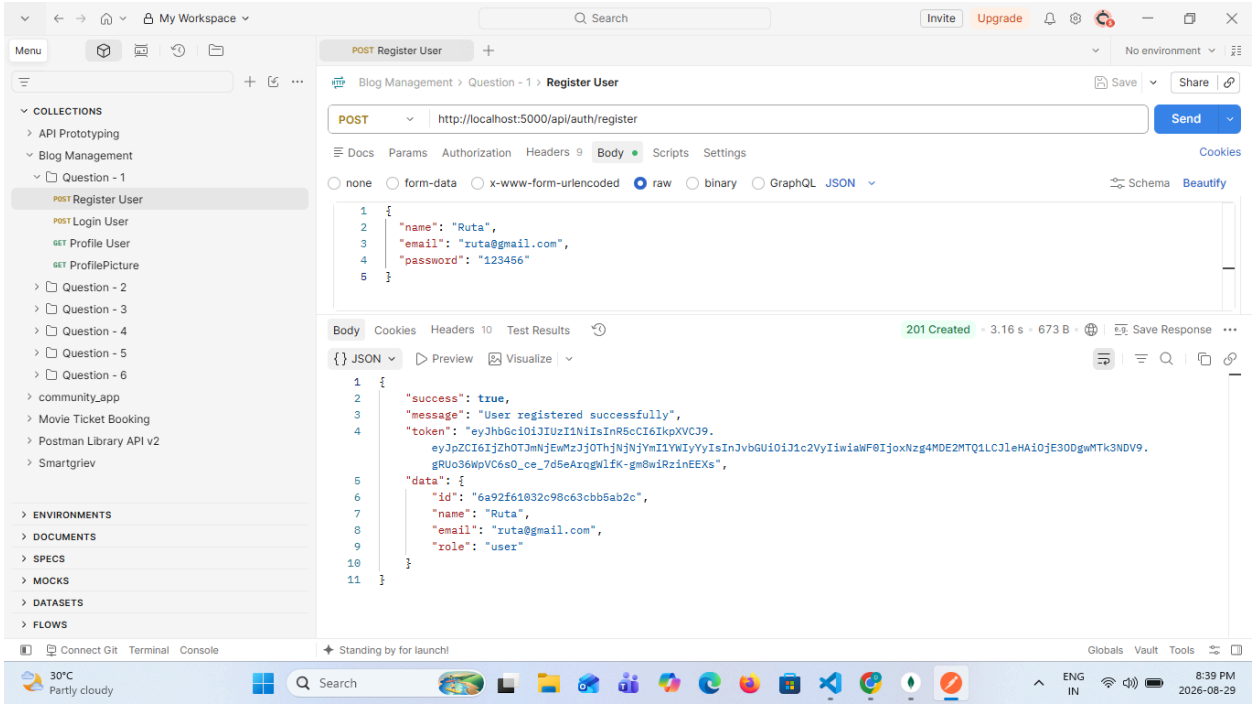
Question 1
Practical Assignment: Blog Management REST API with Authentication (Node.js, Express,
MongoDB, EJS)
Scenario: A startup wants to build the backend and a basic server-rendered interface for a personal blogging platform where registered users can sign up, log in, and manage their own blog posts. Only the author of a post (or an admin) should be able to edit or delete it. You have been asked to build this system.

Build an application using Node.js, Express.js, MongoDB, Mongoose, and EJS (for rendering views) that implements the following:

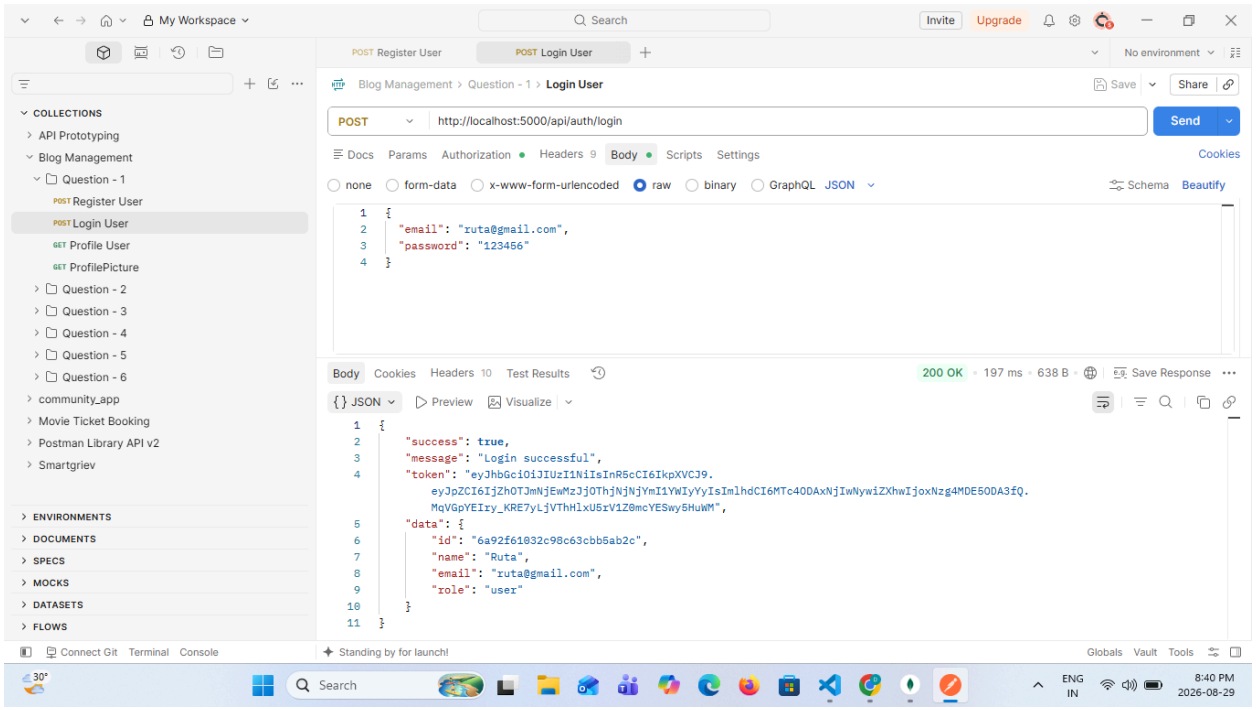
1. User Authentication

● POST /api/auth/register – Register a new user (name, email, password) with the password hashed using bcrypt.

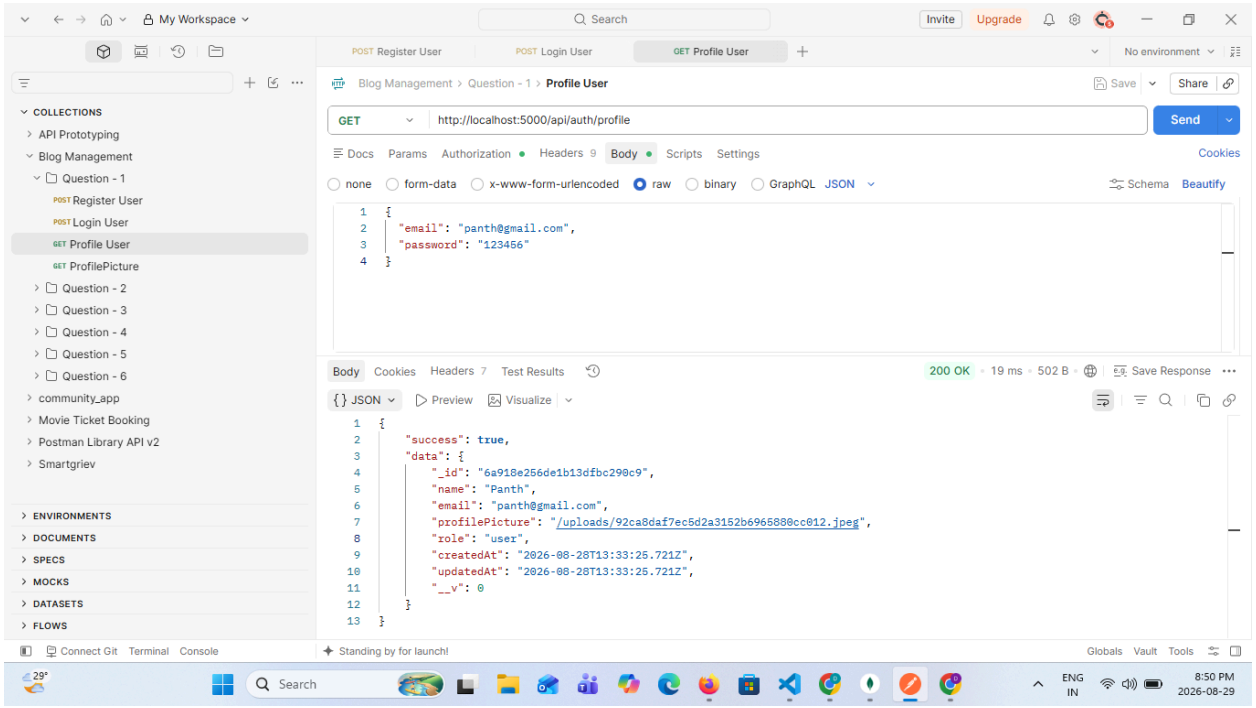

File Edit Selection View Go Run
Explorer
A2_Q-1
blog-management
controllers
authController.js
postController.js
middleware
adminMiddleware.js
authMiddleware.js
errorMiddleware.js
uploadMiddleware.js
validationMiddleware.js
models
Post.js
User.js
node_modules
public
routes
authRoutes.js
postRoutes.js
viewRoutes.js
uploads
views
.env
.env.example
.gitignore
Outline
Timeline
README.md
User.js
authRoutes.js
blog-management > routes > authRoutes.js > ...
upload.single("profilePicture"),
uploadProfilePicture,
);
router.get("/register", (req, res) => {
res.render("register", {
error: null,
});
});
router.post("/register", browserRegister);
module.exports = router;
Problems Output Debug Console Terminal Ports
node - blog-management
PS D:\05_DhruvishaBhaliya_304_A2\A2_Q-1\blog-management> npm run dev
> blog-management@1.0.0 dev
> nodemon server.js
[nodemon] 3.1.14
[nodemon] to restart at any time, enter `rs`
[nodemon] watching path(s): *.*
[nodemon] watching extensions: js,mjs,cjs,json
[nodemon] starting `node server.js`
injected env (6) from .env // tip: encrypted .env [www.dotenvx.com]
Server running on http://localhost:5000
MongoDB connected successfully
Ln 17, Col 1 (40 selected) Spaces: 2 UTF-8 CRLF JavaScript Go Live Prettier
30°C
Partly cloudy
Search
ENG
IN
8:36 PM
2026-08-29


My Workspace
Search
Invite
Upgrade
Menu
POST Register User
No environment
Blog Management > Question - 1 > Register User
Save
Share
POST
http://localhost:5000/api/auth/register
Send
Docs Params Authorization Headers 9 Body Scripts Settings
Cookies
none form-data x-www-form-urlencoded raw binary GraphQL JSON
Schema Beautify
{
"name": "Ruta",
"email": "ruta@gmail.com",
"password": "123456"
}
COLLECTIONS
API Prototyping
Blog Management
Question - 1
POST Register User
POST Login User
GET Profile User
GET ProfilePicture
Question - 2
Question - 3
Question - 4
Question - 5
Question - 6
community_app
Movie Ticket Booking
Postman Library API v2
Smartgriev
ENVIRONMENTS
DOCUMENTS
SPECS
MOCKS
DATASETS
FLOWS
Body Cookies Headers 10 Test Results
201 Created 3.16 s 673 B Save Response
{ } JSON Preview Visualize
{
"success": true,
"message": "User registered successfully",
"token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.
eyJpZCI6IjZhOTJmNjEwMzJjOThjNjNjYmI1YWIyYyIsInJvbGUiOiJ1c2VyIiwiaWF0IjoxNzg4MDE2MTQ1LCJleHAiOjE3ODgwMTk3NDV9.
gRUo36WpVC6sO_ce_7d5eArqgWlfK-gm8wiRzinEEXs",
"data": {
"id": "6a92f61032c98c63cbb5ab2c",
"name": "Ruta",
"email": "ruta@gmail.com",
"role": "user"
}
}
Connect Git Terminal Console
Standing by for launch!
Globals Vault Tools
30°C
Partly cloudy
Search
ENG
IN
8:39 PM
2026-08-29

- POST /api/auth/login – Authenticate a user and return a JWT token.

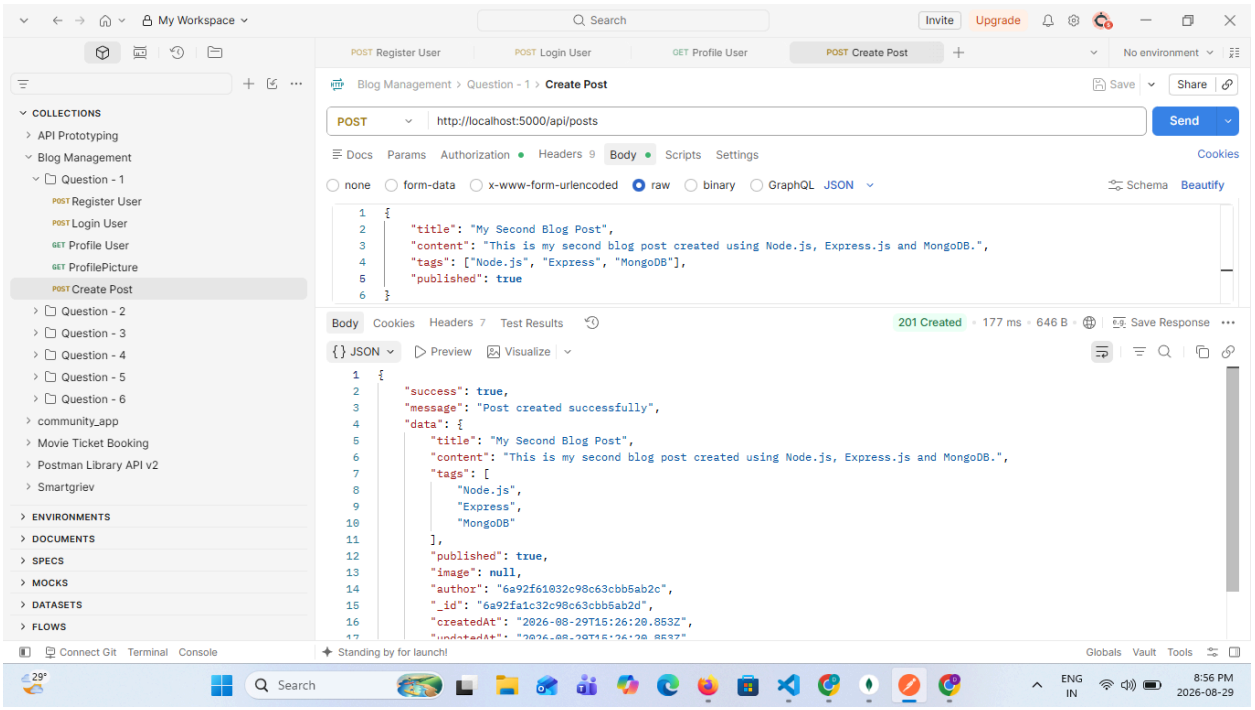


- GET /api/auth/profile – A protected route that returns the logged-in user's profile.

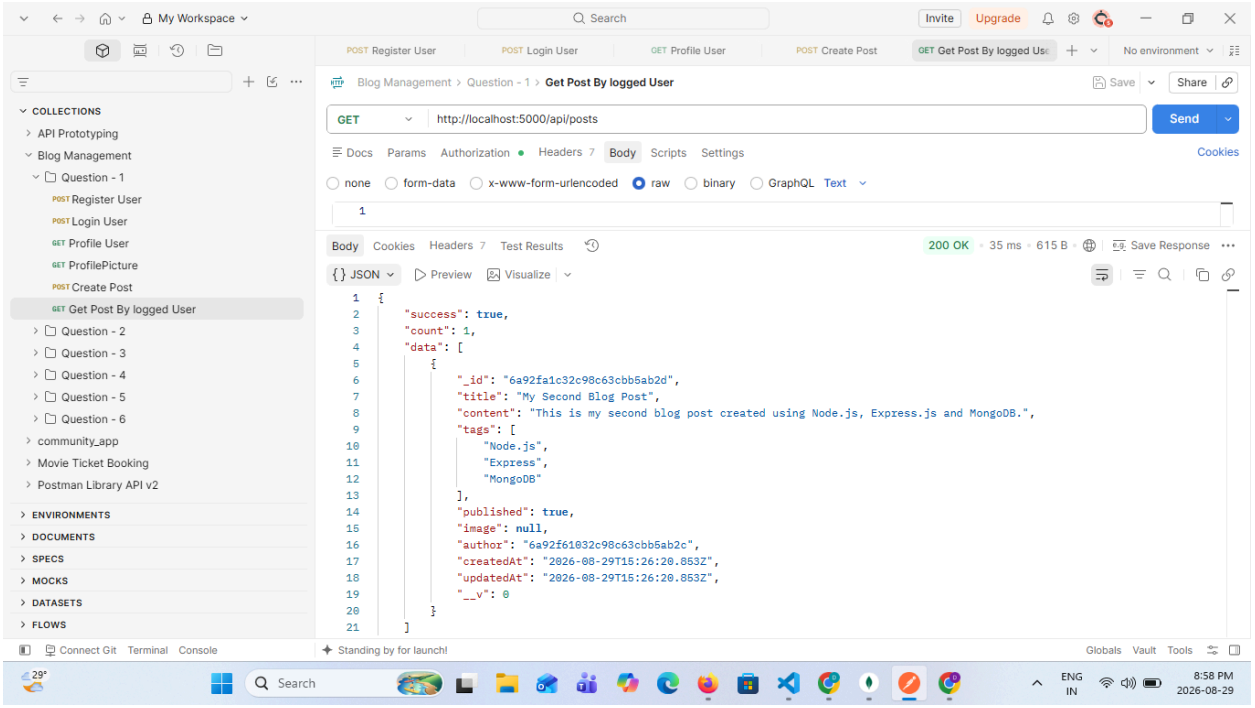


2. Blog Post CRUD API (protected routes, linked to the logged-in author)

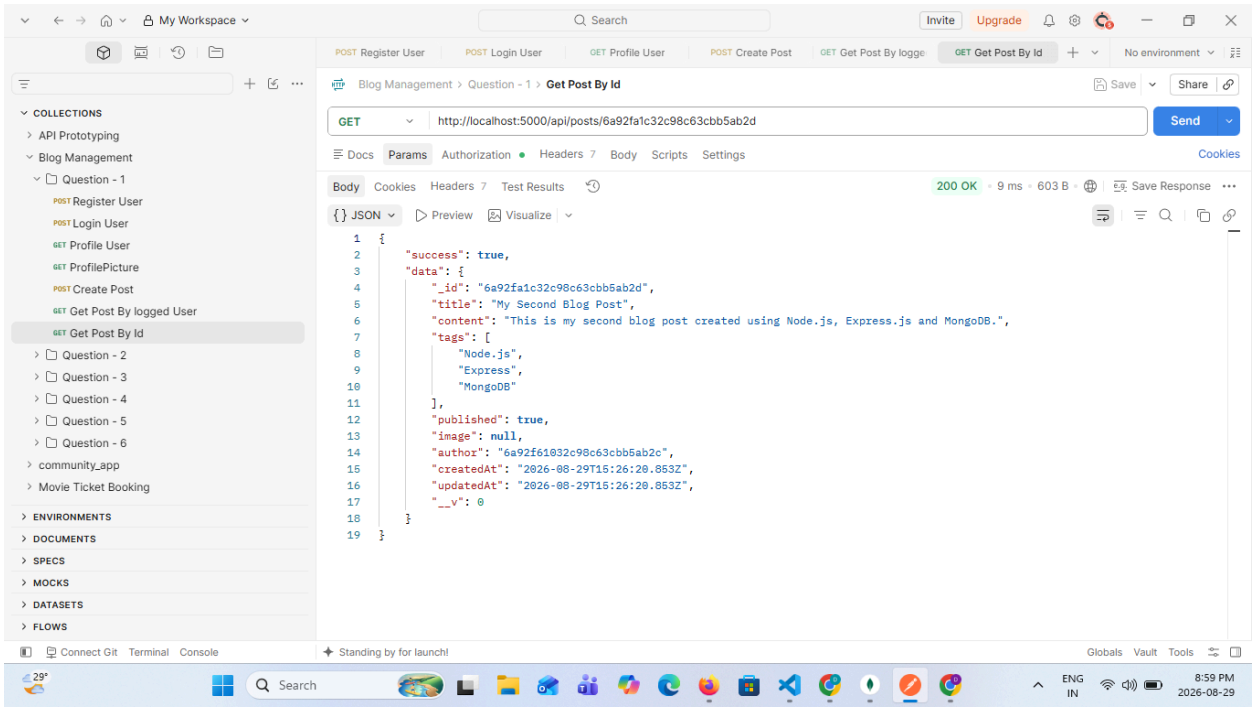
● POST /api/posts – Create a new blog post (title, content, tags, published status).



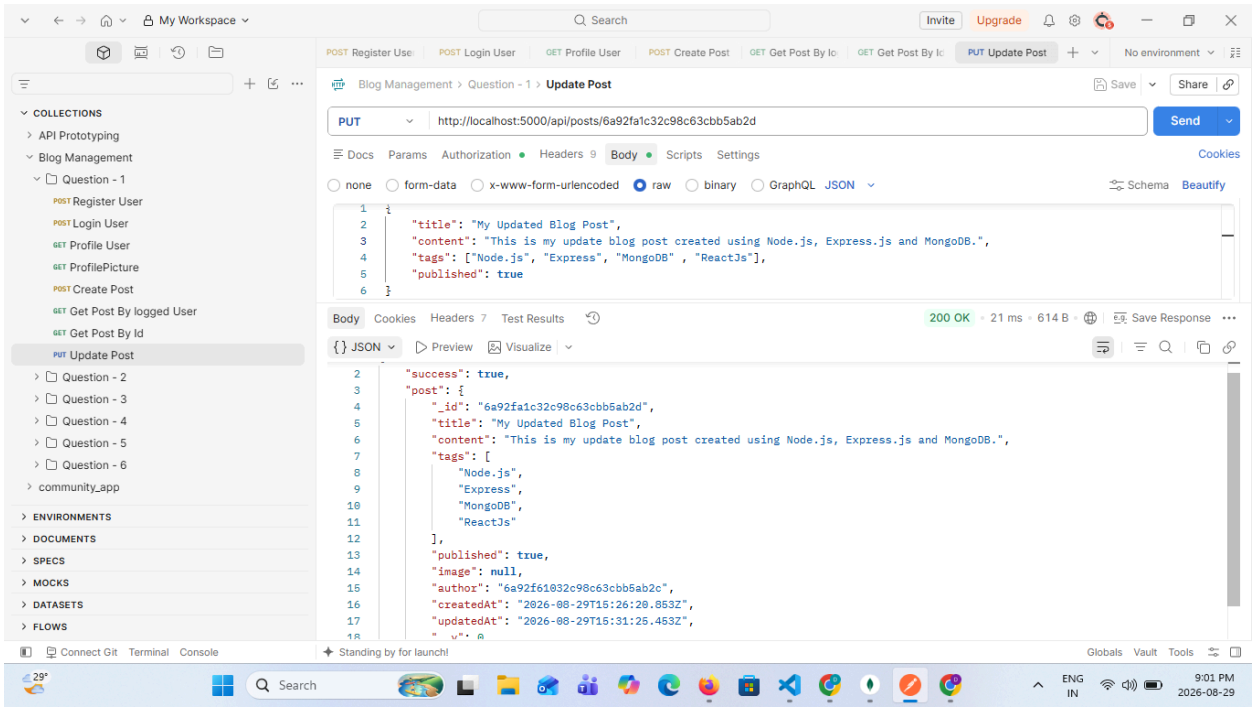
● GET /api/posts – Retrieve all blog posts belonging to the logged-in user.



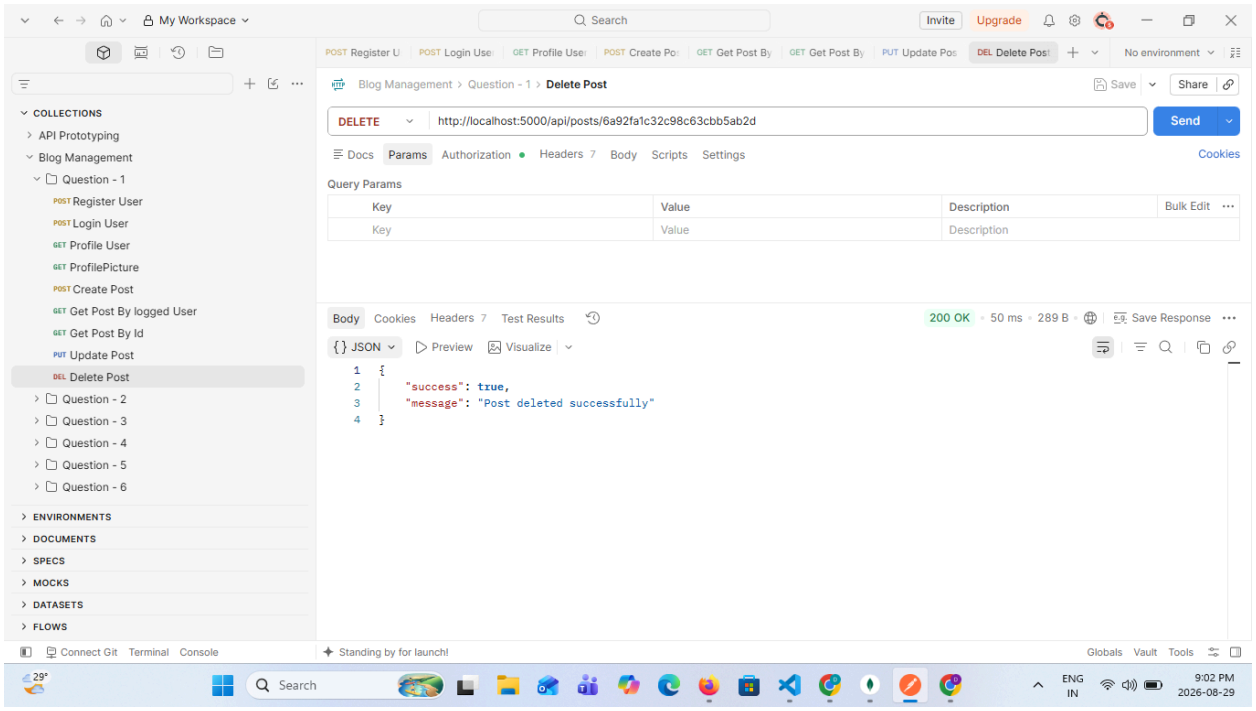
● GET /api/posts/:id – Retrieve a single blog post by ID.



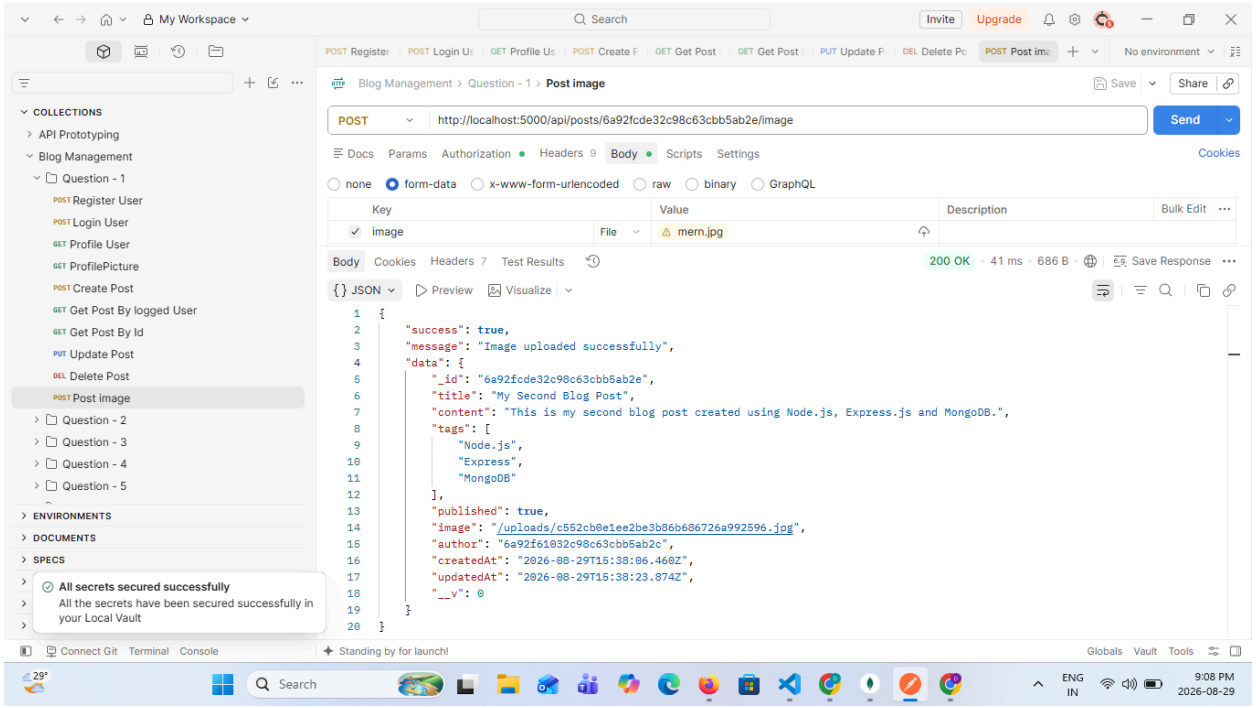
● PUT /api/posts/:id – Update a blog post by ID (only the author or an admin can update).



● DELETE /api/posts/:id – Delete a blog post by ID (only the author or an admin can delete).

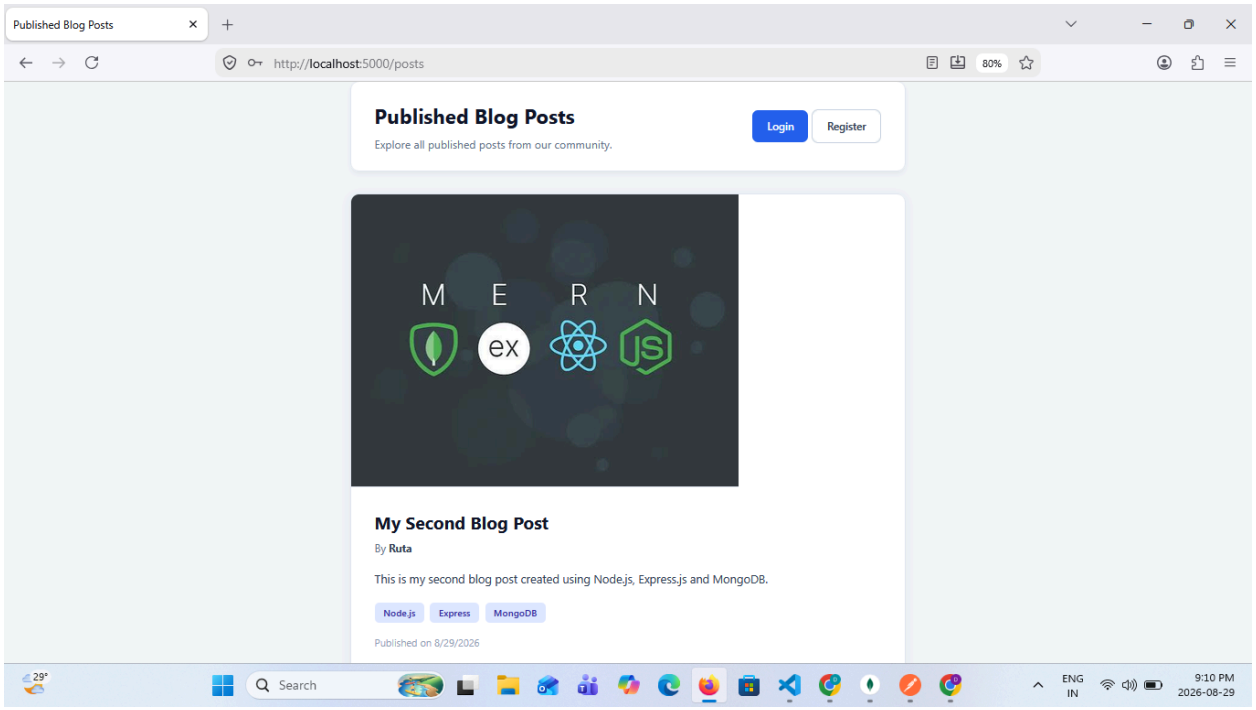


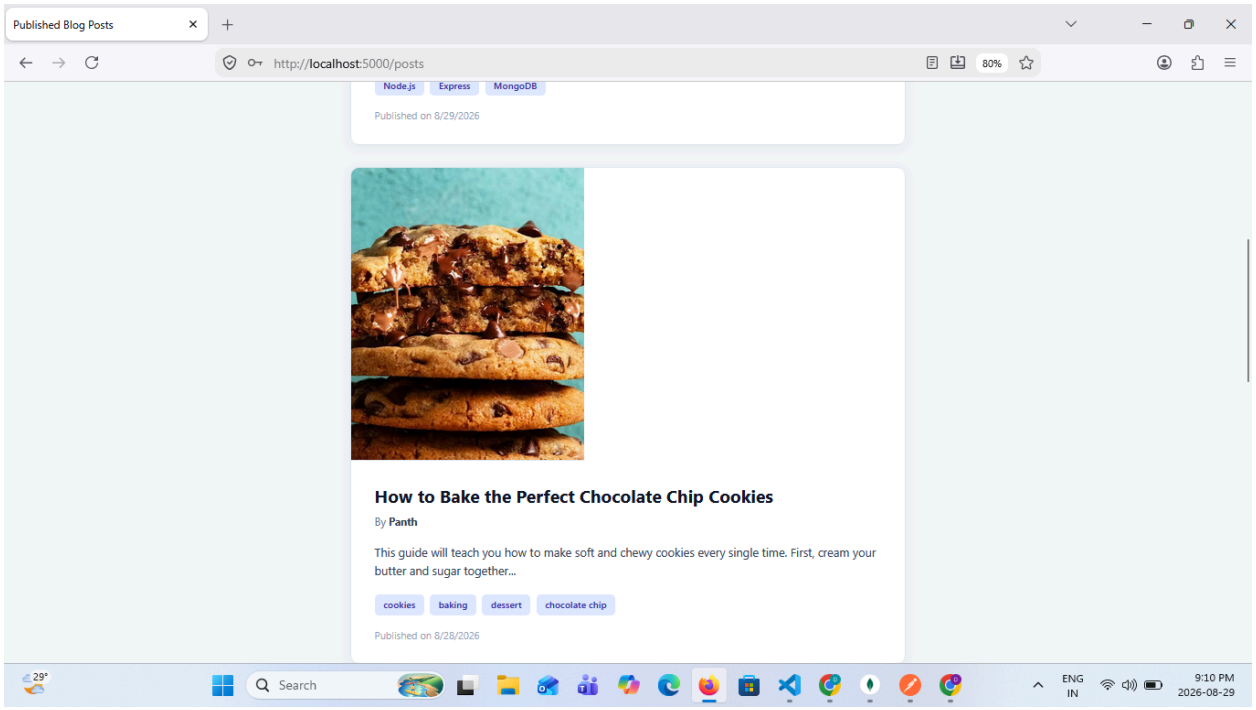
● POST /api/posts/:id/image – Upload or replace a featured image for a blog post (multipart/form-data) using Multer.

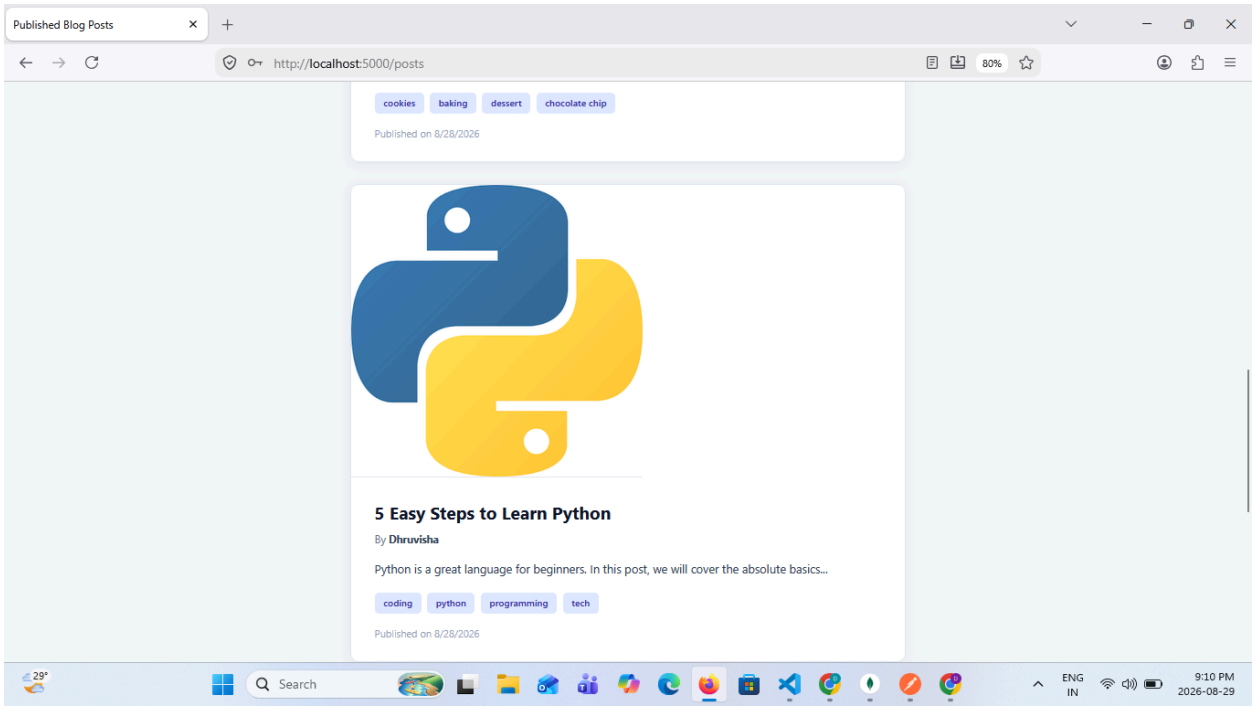


3. EJS Views

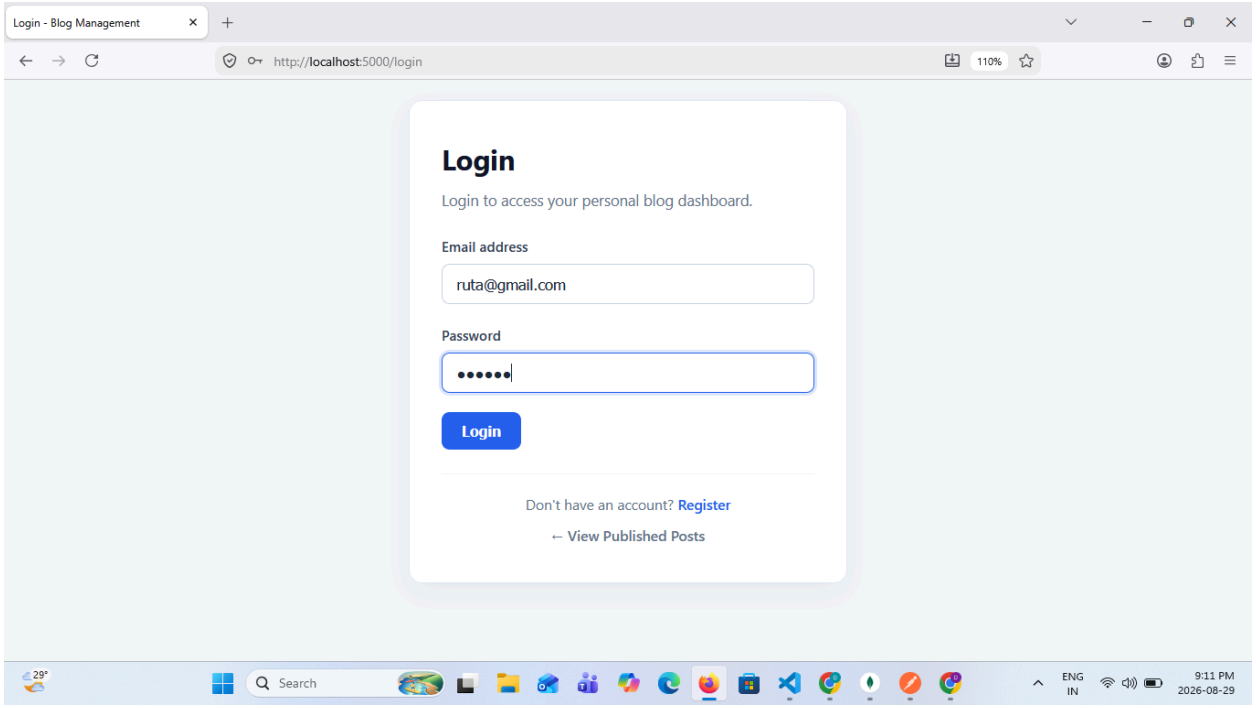
● Render an EJS page (views/posts.ejs) that displays all published blog posts in a styled list.

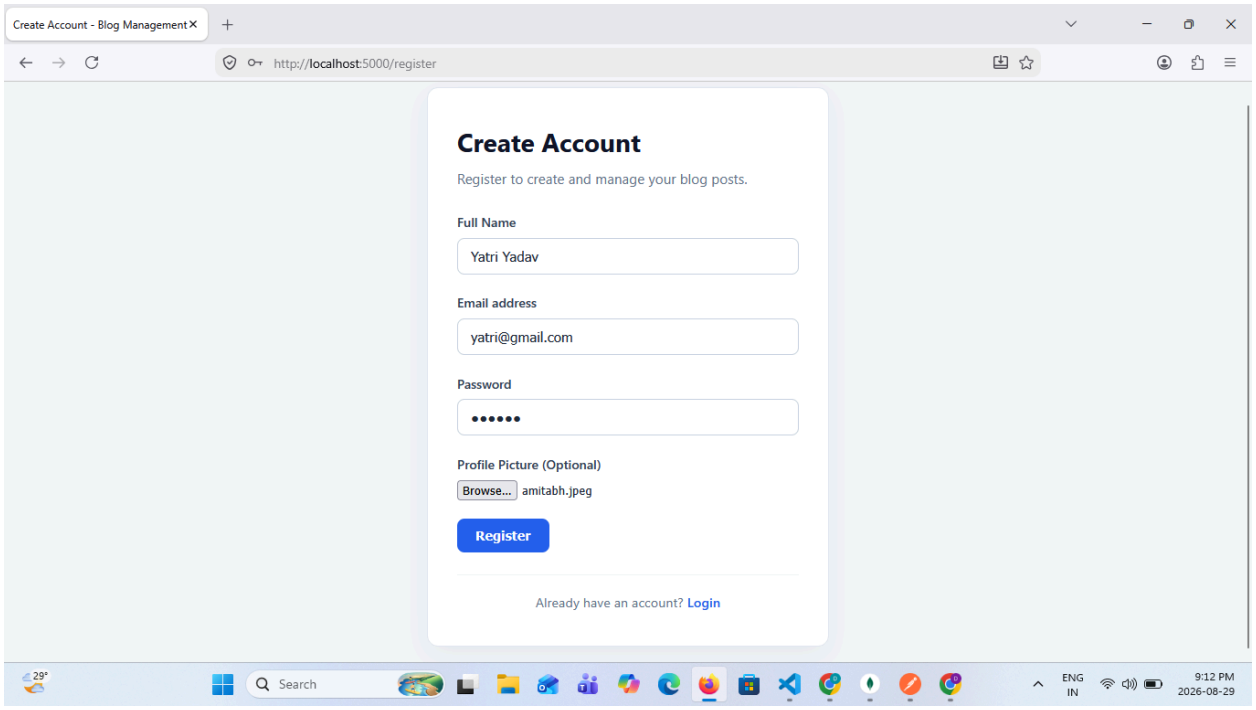




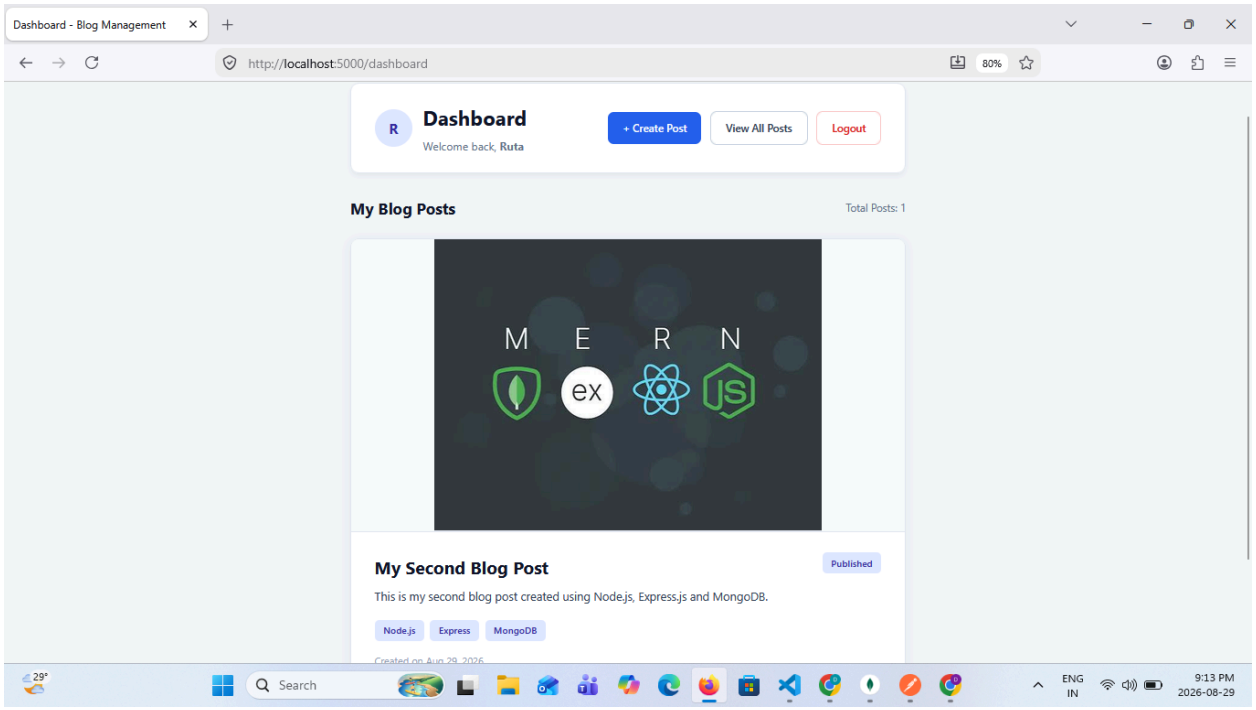


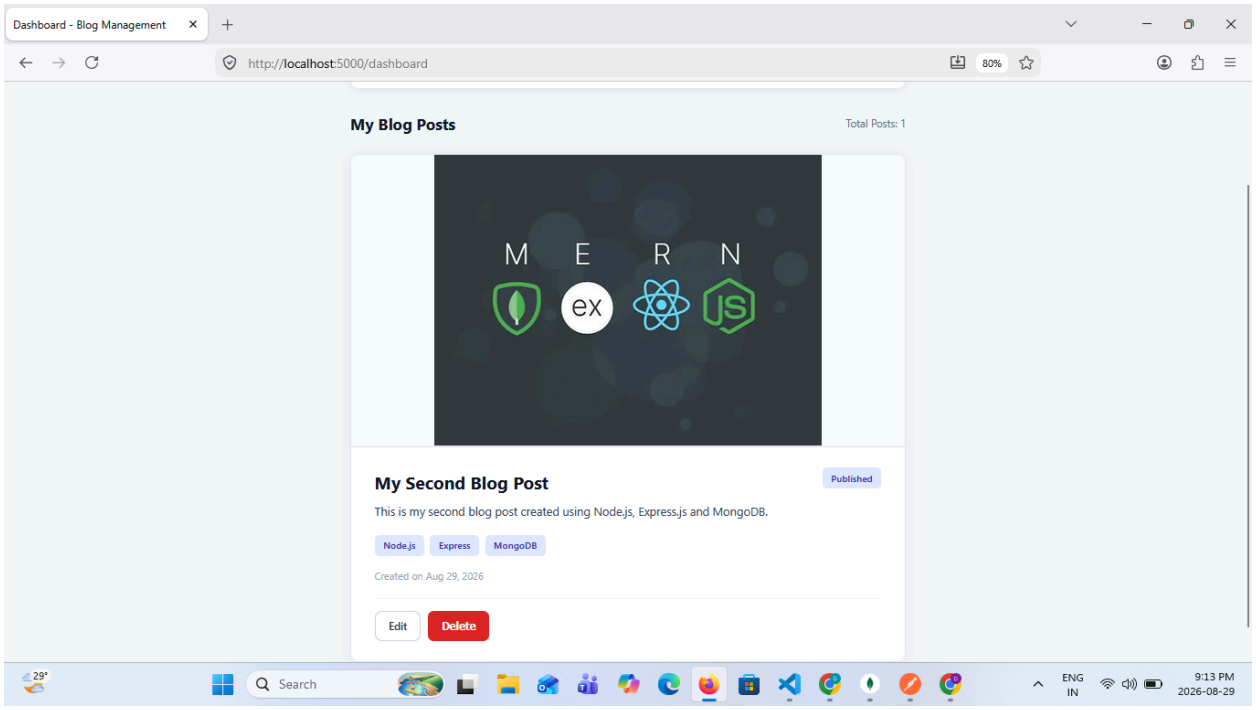
● Render an EJS page (views/login.ejs) with a login form that submits to your login route.





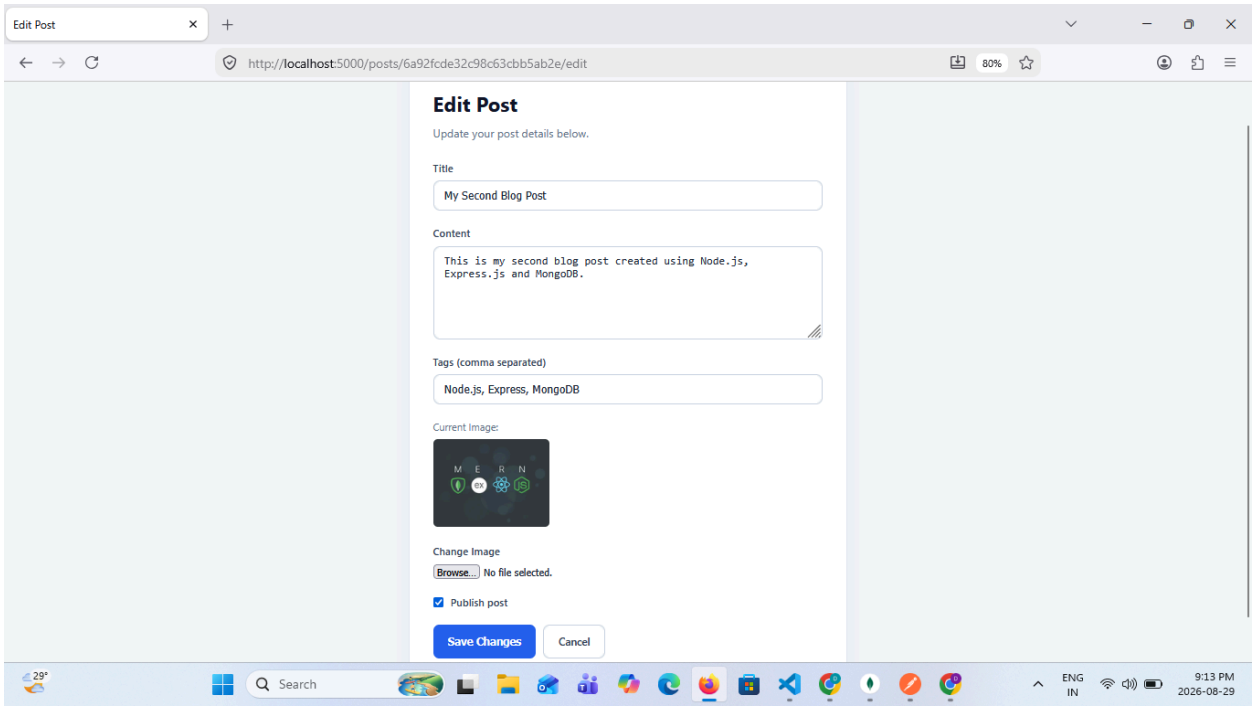
● Render an EJS page (views/dashboard.ejs, protected) showing the logged-in user's own posts with edit/delete links, visible only after login.

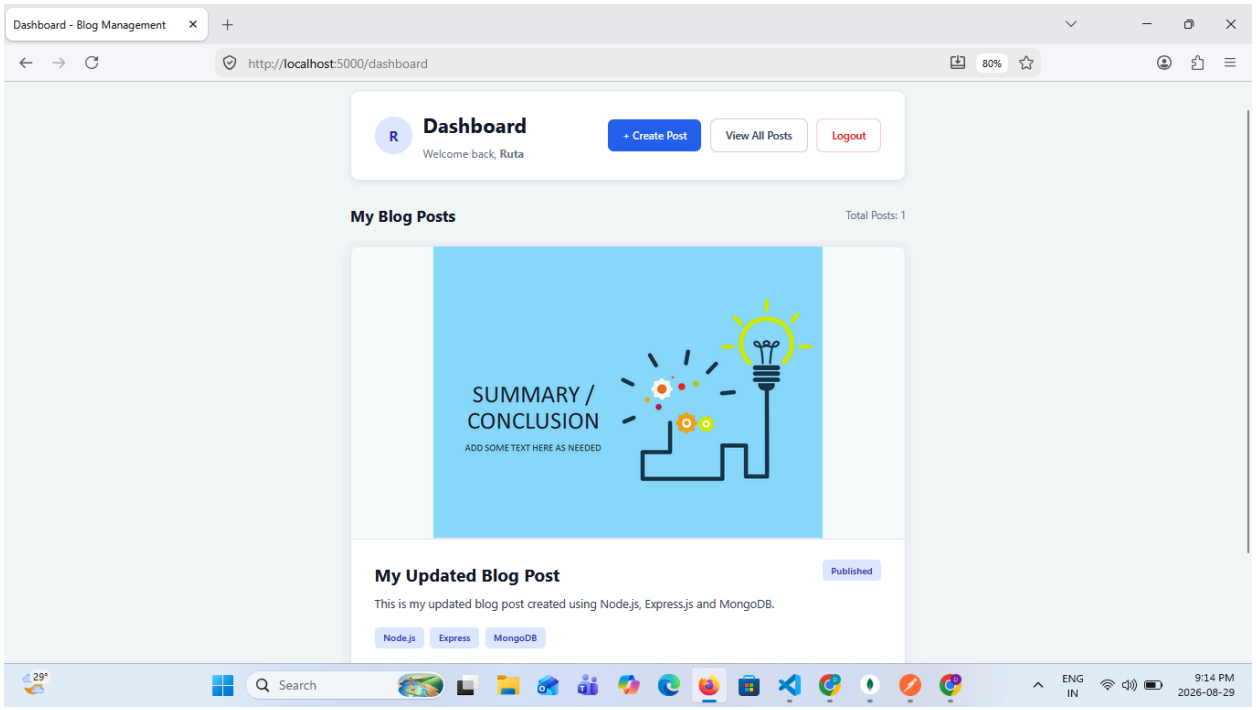




4. File Uploading

● Use Multer to handle multipart/form-data uploads for a blog post's featured image and/or a user's profile picture.
● Restrict uploads to image MIME types (jpeg, png, webp) and enforce a maximum file size (e.g.,2MB); reject and return a clear error for anything else.
● Store uploaded files in a local /uploads directory (or a cloud service such as Cloudinary/AWS S3) and save only the resulting file path/URL on the Post or User document.
● Serve uploaded images as static assets and render them in views/posts.ejs and views/dashboard.ejs.
● Delete the associated image file from storage when a post (or its image) is deleted or replaced.

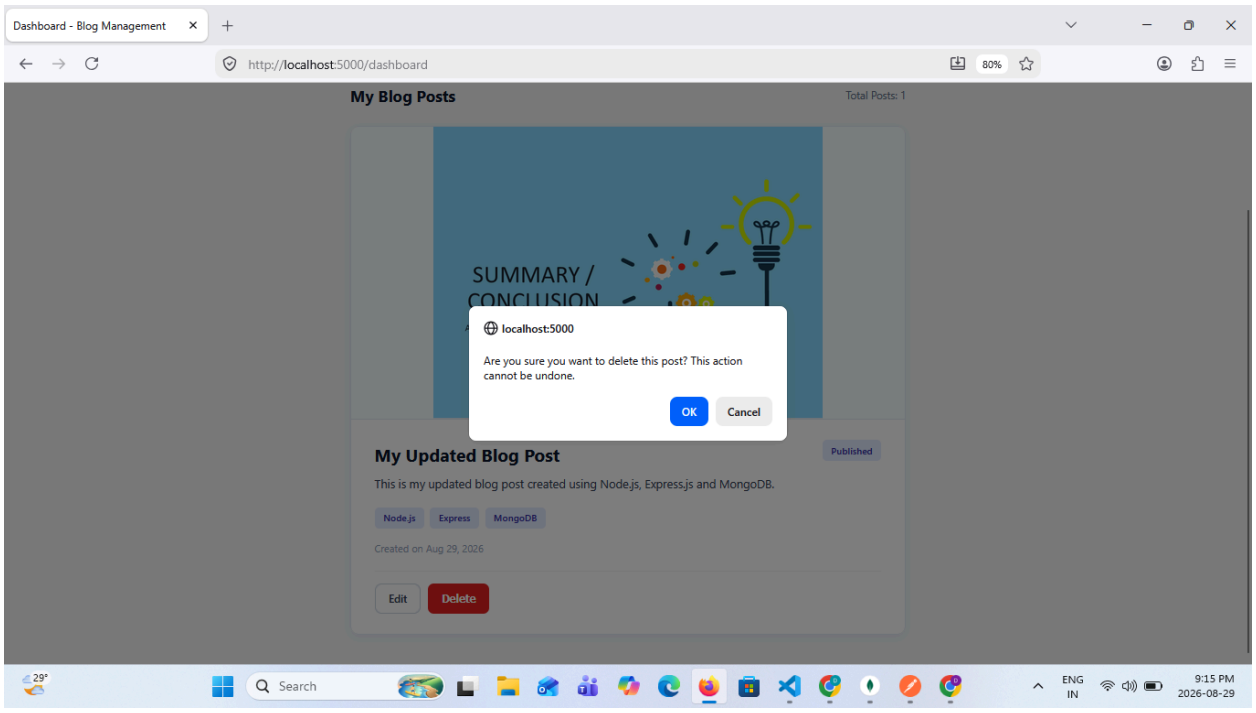




5. Security

● Use Helmet to set secure HTTP headers, and configure CORS to only allow trusted origins.

● Apply rate limiting (e.g., express-rate-limit) on /api/auth/login and /api/auth/register to mitigate brute-force and credential-stuffing attacks.
● Validate and sanitize all request input (e.g., with express-validator or Joi) to guard against NoSQL injection and malformed payloads.
● Keep secrets (JWT signing key, MongoDB URI, upload credentials) in environment variables via dotenv and never commit them to source control (.env.example only).
● Set a reasonable JWT expiry, sign tokens with a strong secret, and reject/refresh appropriately on expiry.
● Validate uploaded file types on the server (not just by extension) and never trust the client-supplied filename when saving to disk.

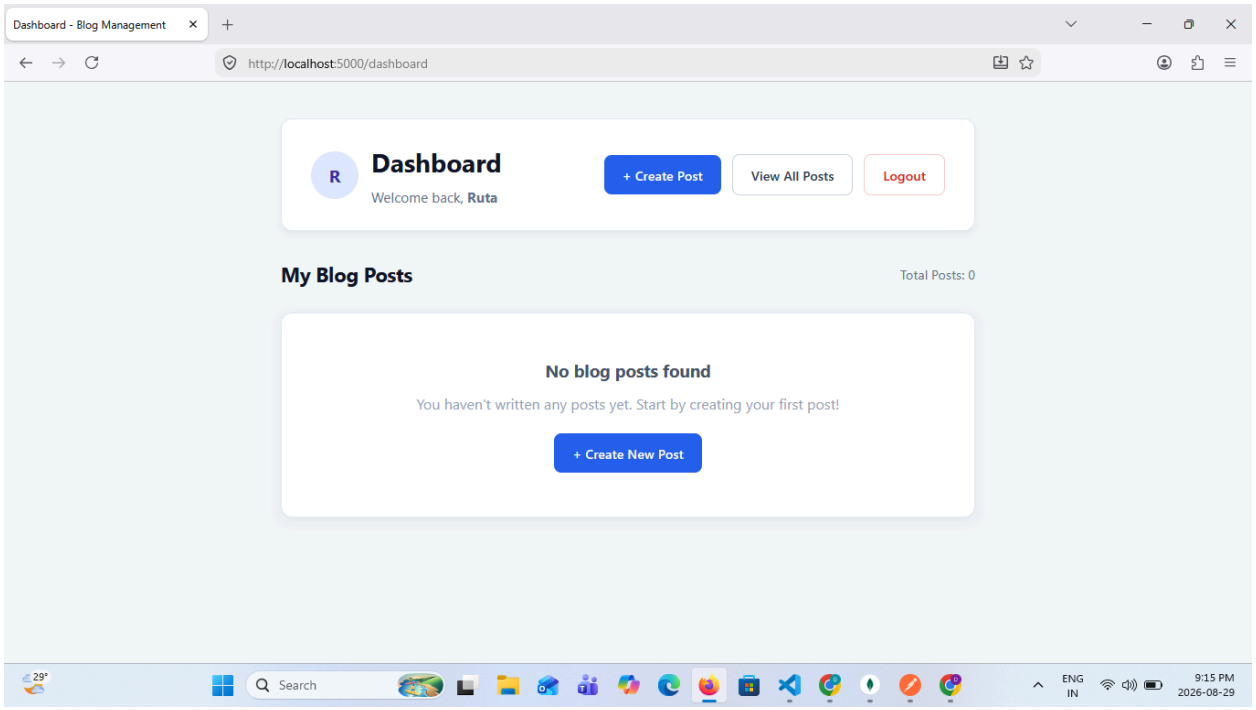


6. Requirements

● Use bcrypt to hash and compare passwords — never store plain text passwords.
● Use JWT middleware to protect all post-related API and dashboard routes.
● Use Mongoose schemas to model User and Post, with each post referencing its author via ObjectId and each user having a role field (user/admin).
● Ensure a user can only view/edit/delete their own posts, unless they are an admin.
● Return appropriate HTTP status codes and consistent JSON responses for the API routes.
● Never trust client input for authorization decisions — always re-check ownership/role server-side, even if the UI hides edit/delete links.

● Apply the same Security requirements above (Helmet, rate limiting, input validation, env-based secrets) across both the API and file-upload routes.

Deliverable: A working Node.js/Express project with proper folder structure (models, routes,controllers, middleware, views, uploads), a README.md explaining setup and usage (including a.env.example listing required environment variables), and example API requests (curl or Postman)demonstrating registration, login, all blog post CRUD operations, and image upload, plus screenshots or a demo of the EJS pages.



Create New Post | "Node.js", "Express.js", "Mongo | http://localhost:5000/posts/new | 90%

Create New Post

Publish your thoughts or save a draft for later.

Title

Building a Secure Blog API with Node.js

Content

This blog post explains how Node.js, Express.js, MongoDB, JWT authentication, and bcrypt can be used together to build a secure blog management application. Users can create and manage their own posts while authorization ensures that only the owner or an administrator can modify them.

Tags (comma separated)

"Node.js", "Express.js", "MongoDB", "JWT", "Web Security"

Featured Image

Browse... mern1.jpg

Publish immediately

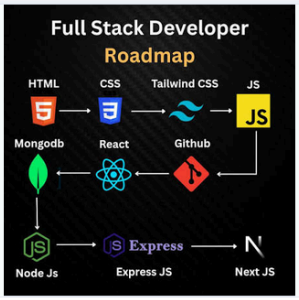
Create Post | Cancel

29° | Search | ENG IN | 9:18 PM 2026-08-29

Dashboard - Blog Management | "Node.js", "Express.js", "Mongo | http://localhost:5000/dashboard | 80%

My Blog Posts

Total Posts: 1



Building a Secure Blog API with Node.js

Published

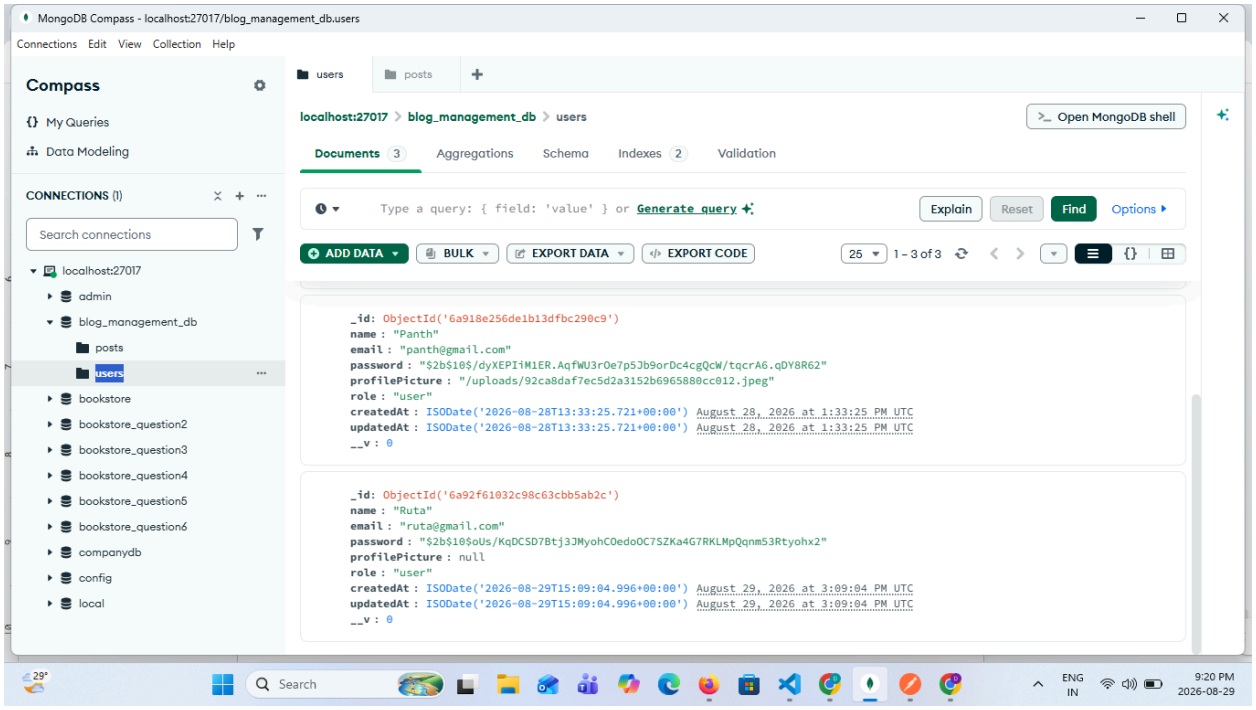
This blog post explains how Node.js, Express.js, MongoDB, JWT authentication, and bcrypt can be used together to build a secure blog management application. Users can create and manage their own posts while authorization ensures that only the owner or an administrator can modify them.

"Node.js" "Express.js" "MongoDB" "JWT" "Web Security"

Created on Aug 29, 2026

Edit | Delete

29° | Search | ENG IN | 9:19 PM 2026-08-29

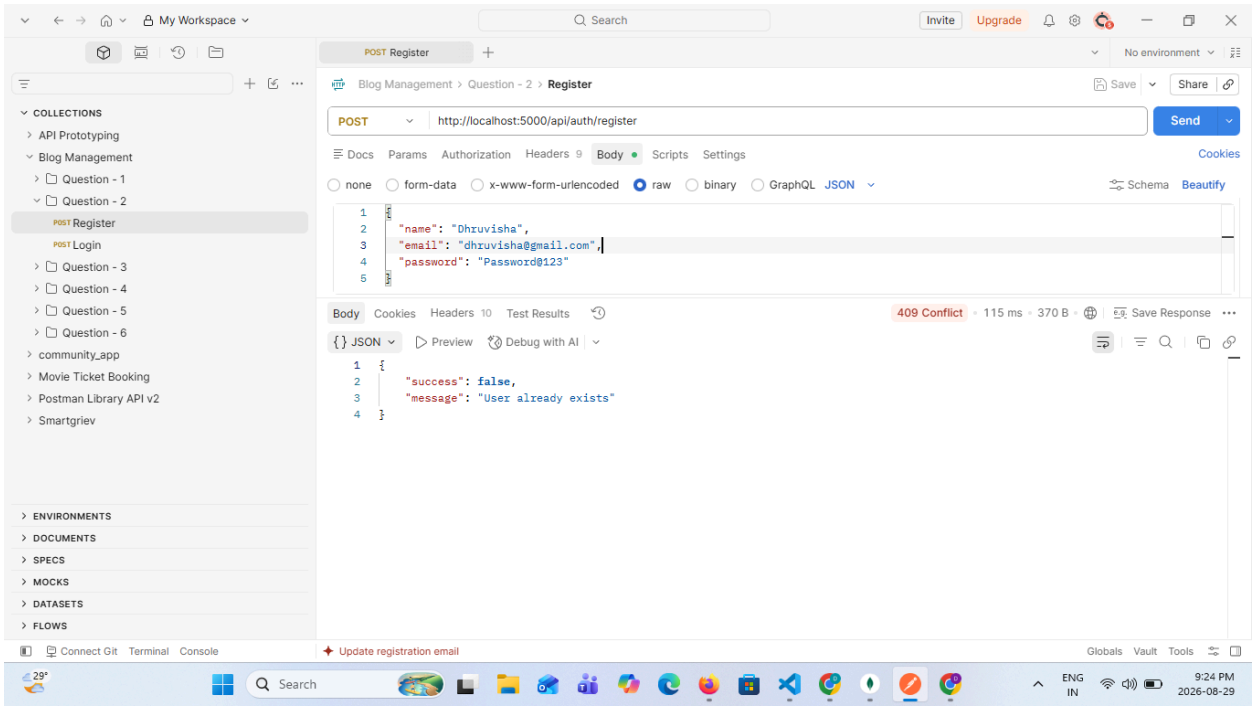


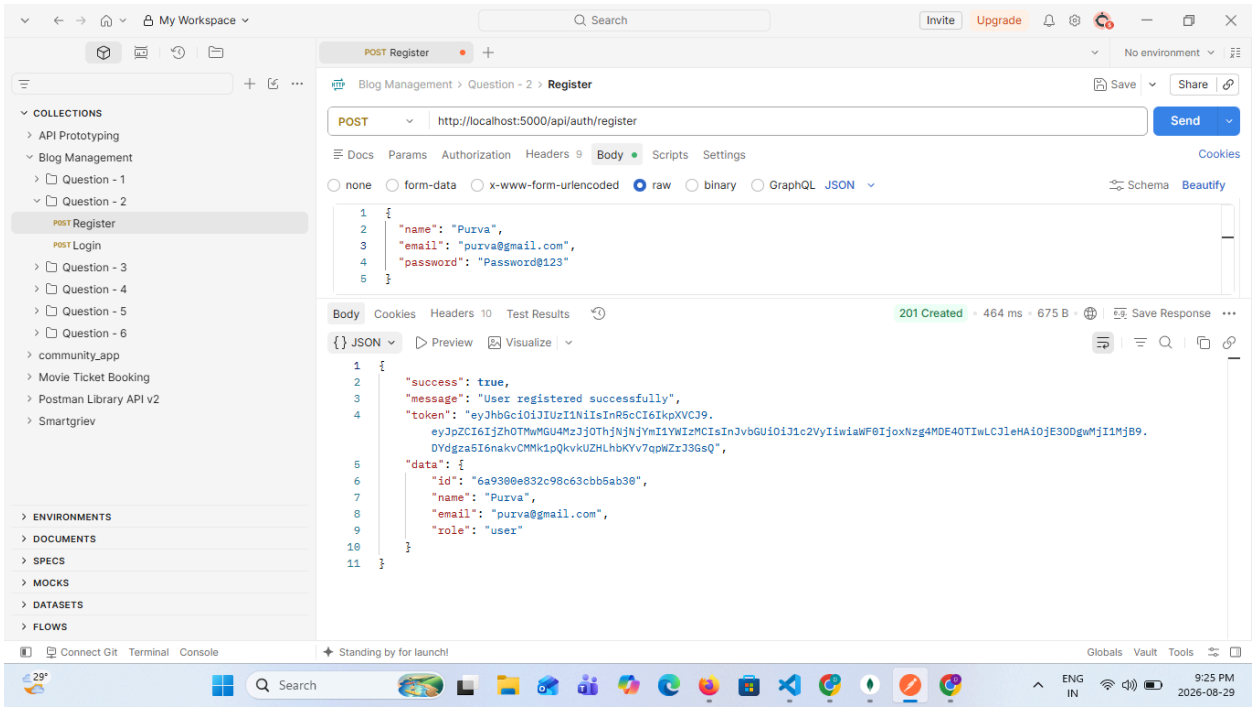
Question 2

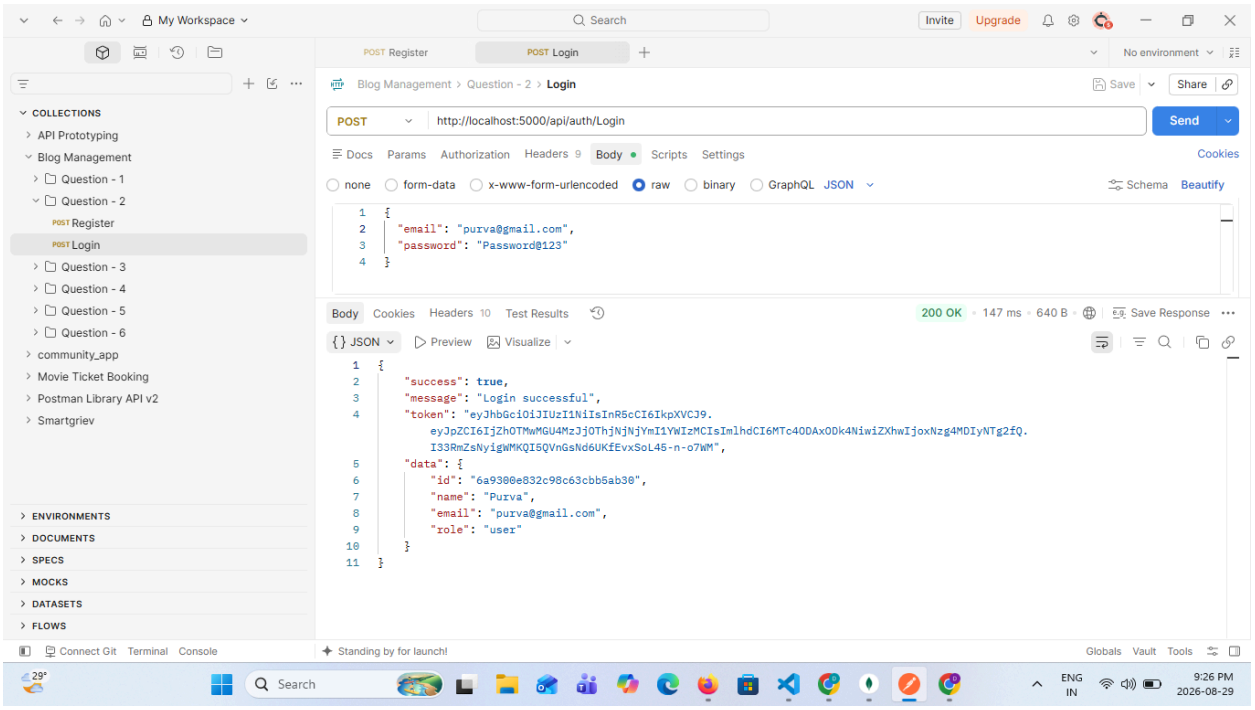
Secure Signup for Bookstore Customers

Scenario: A new customer is creating an account on the online bookstore to start placing orders for books.

Task: Write a program that hashes the customer's password using bcrypt before storing it in MongoDB, and write the corresponding logic to verify the password when the same customer logs in later.


My Workspace
Search
Invite
Upgrade
POST Register
No environment
Blog Management › Question - 2 › Register
Save
Share
POST
http://localhost:5000/api/auth/register
Send
Docs Params Authorization Headers 9 Body Scripts Settings
Cookies
none form-data x-www-form-urlencoded raw binary GraphQL JSON
Schema Beautify
{
"name": "Dhruvisha",
"email": "dhruvisha@gmail.com",
"password": "Password@123"
}
Body Cookies Headers 10 Test Results
409 Conflict 115 ms 370 B Save Response
{ } JSON Preview Debug with AI
{
"success": false,
"message": "User already exists"
}
COLLECTIONS
API Prototyping
Blog Management
Question - 1
Question - 2
POST Register
POST Login
Question - 3
Question - 4
Question - 5
Question - 6
community_app
Movie Ticket Booking
Postman Library API v2
Smartgriev
ENVIRONMENTS
DOCUMENTS
SPECS
MOCKS
DATASETS
FLOWS
Connect Git Terminal Console
Update registration email
Globals Vault Tools
Search
ENG IN
9:24 PM 2026-08-29


My Workspace
Search
Invite
Upgrade
POST Register
No environment
Blog Management › Question - 2 › Register
Save
Share
POST
http://localhost:5000/api/auth/register
Send
Docs Params Authorization Headers 9 Body Scripts Settings
Cookies
none form-data x-www-form-urlencoded raw binary GraphQL JSON
Schema Beautify
{
"name": "Purva",
"email": "purva@gmail.com",
"password": "Password@123"
}
Body Cookies Headers 10 Test Results
201 Created 464 ms 675 B Save Response
{ } JSON Preview Visualize
{
"success": true,
"message": "User registered successfully",
"token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.
eyJpZCI6IjZhOTMwMGU4MzJjOThjNjNjYmI1YWIzMCIsInJvbGUiOiJ1c2VyIiwiaWF0IjoxNzg4MDE4OTIwLCJleHAiOjE3ODgwMjI1MjB9.
DYdgza5I6nakvCMMk1pQkvkUZHLhbKYv7qpWZrJ3GsQ",
"data": {
"id": "6a9300e832c98c63cbb5ab30",
"name": "Purva",
"email": "purva@gmail.com",
"role": "user"
}
}
COLLECTIONS
API Prototyping
Blog Management
Question - 1
Question - 2
POST Register
POST Login
Question - 3
Question - 4
Question - 5
Question - 6
community_app
Movie Ticket Booking
Postman Library API v2
Smartgriev
ENVIRONMENTS
DOCUMENTS
SPECS
MOCKS
DATASETS
FLOWS
Connect Git Terminal Console
Standing by for launch!
Globals Vault Tools
Search
ENG IN
9:25 PM 2026-08-29

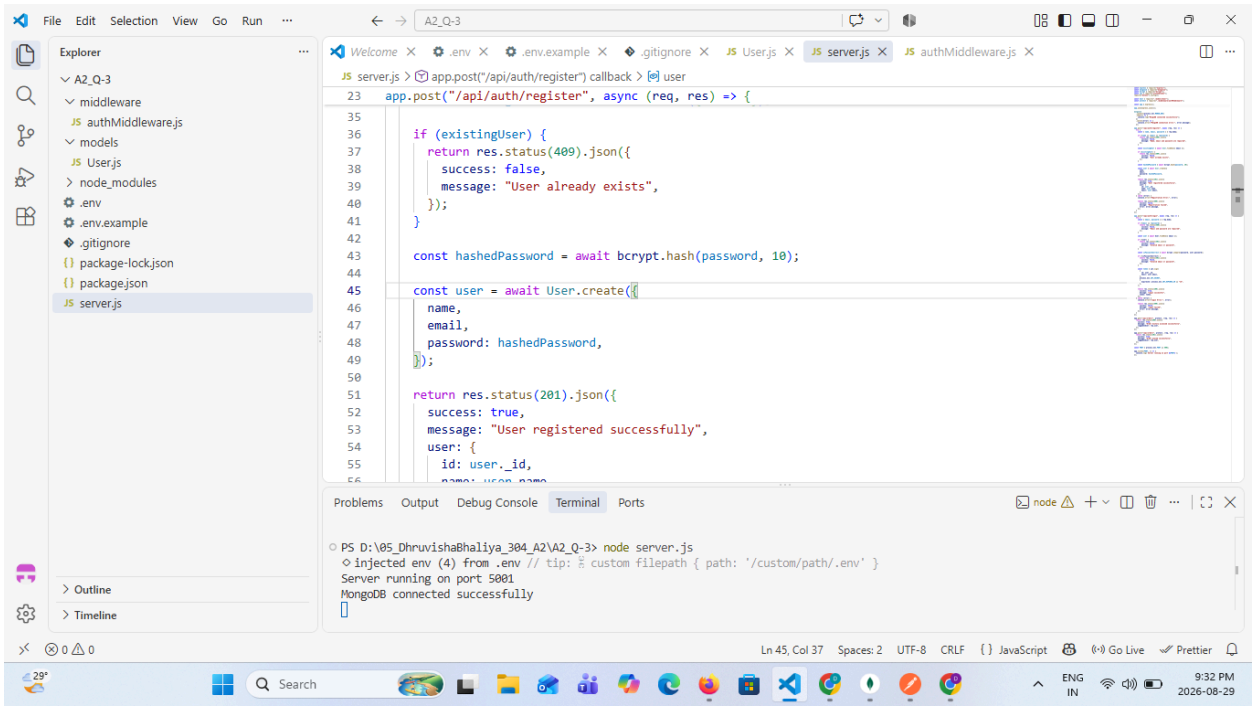


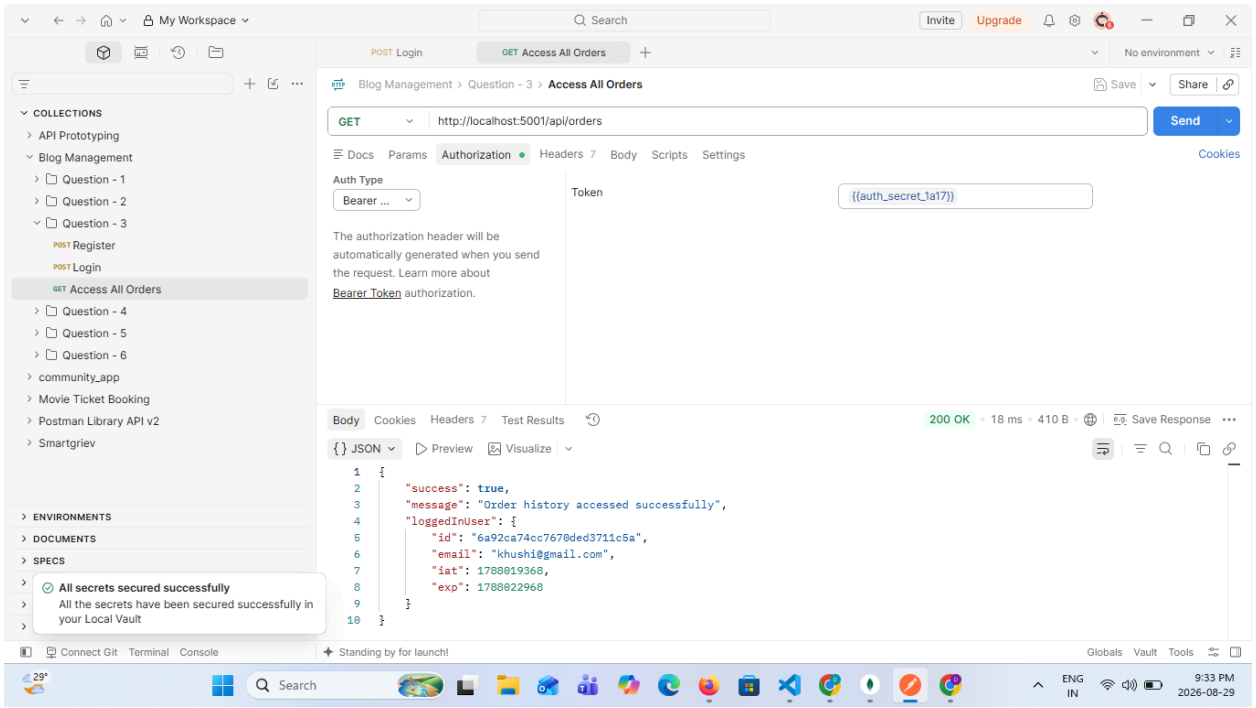
Question 3

Issuing a Digital Membership Token on Login

Scenario: Once a customer logs in successfully, the store must issue them a token (their digital membership pass) so they don't need to log in again for every request, such as viewing their orders or checking out.

Task: Write a program to generate a JWT token on successful login, and a middleware to verify this token before allowing access to any protected route (e.g., placing an order or viewing order history).


File Edit Selection View Go Run
A2_Q-3
Explorer
A2_Q-3
middleware
authMiddleware.js
models
User.js
node_modules
.env
.env.example
.gitignore
package-lock.json
package.json
server.js
Outline
Timeline
Welcome
.env
.env.example
.gitignore
User.js
server.js
authMiddleware.js
server.js > app.post("/api/auth/register") callback > user
23 app.post("/api/auth/register", async (req, res) => {
35
36 if (existingUser) {
37 return res.status(409).json({
38 success: false,
39 message: "User already exists",
40 });
41 }
42
43 const hashedPassword = await bcrypt.hash(password, 10);
44
45 const user = await User.create({
46 name,
47 email,
48 password: hashedPassword,
49 });
50
51 return res.status(201).json({
52 success: true,
53 message: "User registered successfully",
54 user: {
55 id: user._id,
Problems Output Debug Console Terminal Ports
node
PS D:\05_DhruvishaBhaliya_304_A2\A2_Q-3> node server.js
injected env (4) from .env // tip: custom filepath { path: '/custom/path/.env' }
Server running on port 5001
MongoDB connected successfully
Ln 45, Col 37 Spaces: 2 UTF-8 CRLF JavaScript Go Live Prettier
Search
ENG
IN
9:32 PM
2026-08-29

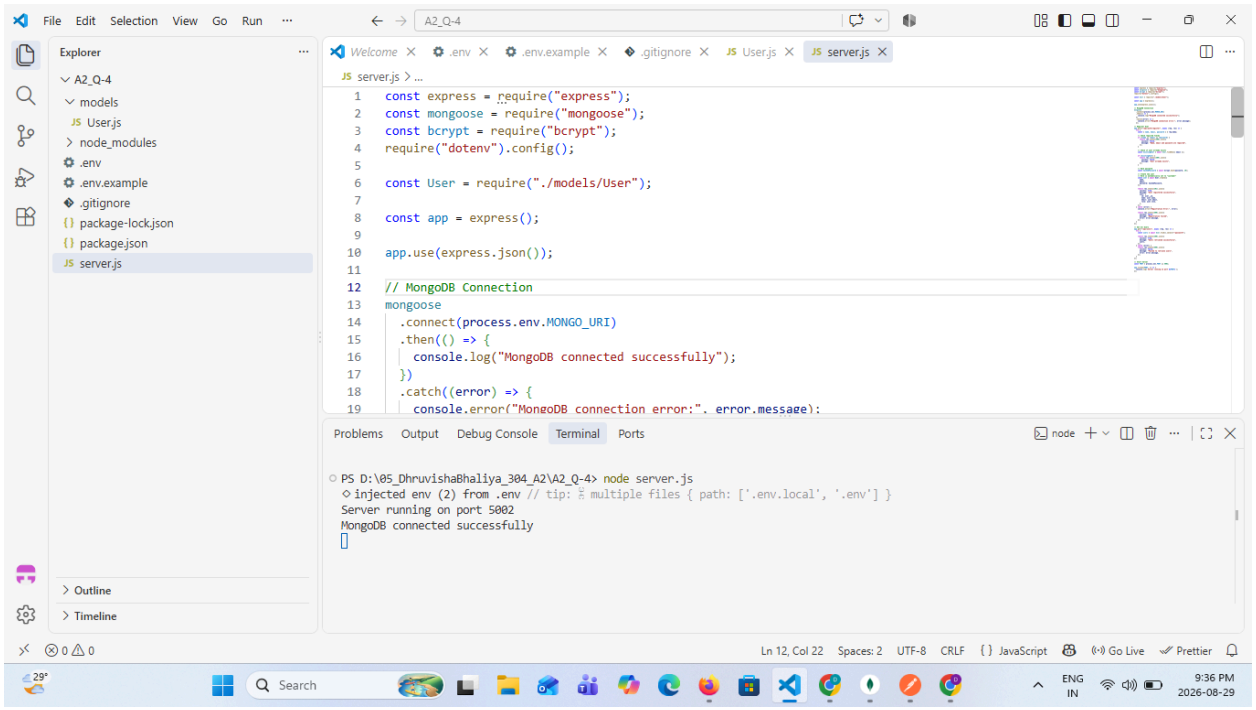

My Workspace
Search
Invite
Upgrade
POST Login
GET Access All Orders
No environment
COLLECTIONS
API Prototyping
Blog Management
Question - 1
Question - 2
Question - 3
POST Register
POST Login
GET Access All Orders
Question - 4
Question - 5
Question - 6
community_app
Movie Ticket Booking
Postman Library API v2
Smartgriev
ENVIRONMENTS
DOCUMENTS
SPECS
Blog Management > Question - 3 > Access All Orders
Save
Share
GET
http://localhost:5001/api/orders
Send
Docs Params Authorization Headers 7 Body Scripts Settings
Cookies
Auth Type
Bearer ...
The authorization header will be automatically generated when you send the request. Learn more about Bearer Token authorization.
Token
{{auth_secret_1a17}}
Body Cookies Headers 7 Test Results
200 OK 18 ms 410 B
Save Response
{} JSON
Preview
Visualize
{
"success": true,
"message": "Order history accessed successfully",
"loggedInUser": {
"id": "6a92ca74cc7670ded3711c5a",
"email": "khushi@gmail.com",
"iat": 1788019368,
"exp": 1788022968
}
}
All secrets secured successfully
All the secrets have been secured successfully in your Local Vault
Connect Git Terminal Console
Standing by for launch!
Globals Vault Tools
Search
ENG
IN
9:33 PM
2026-08-29

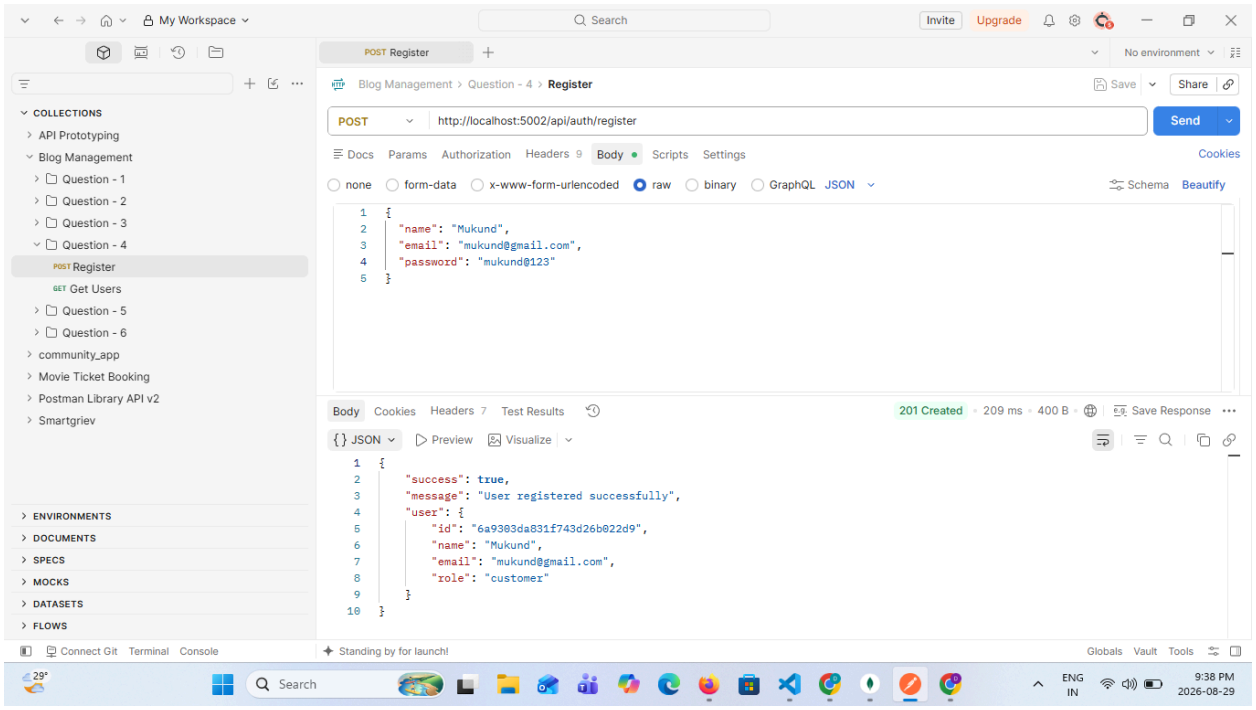
Question 4

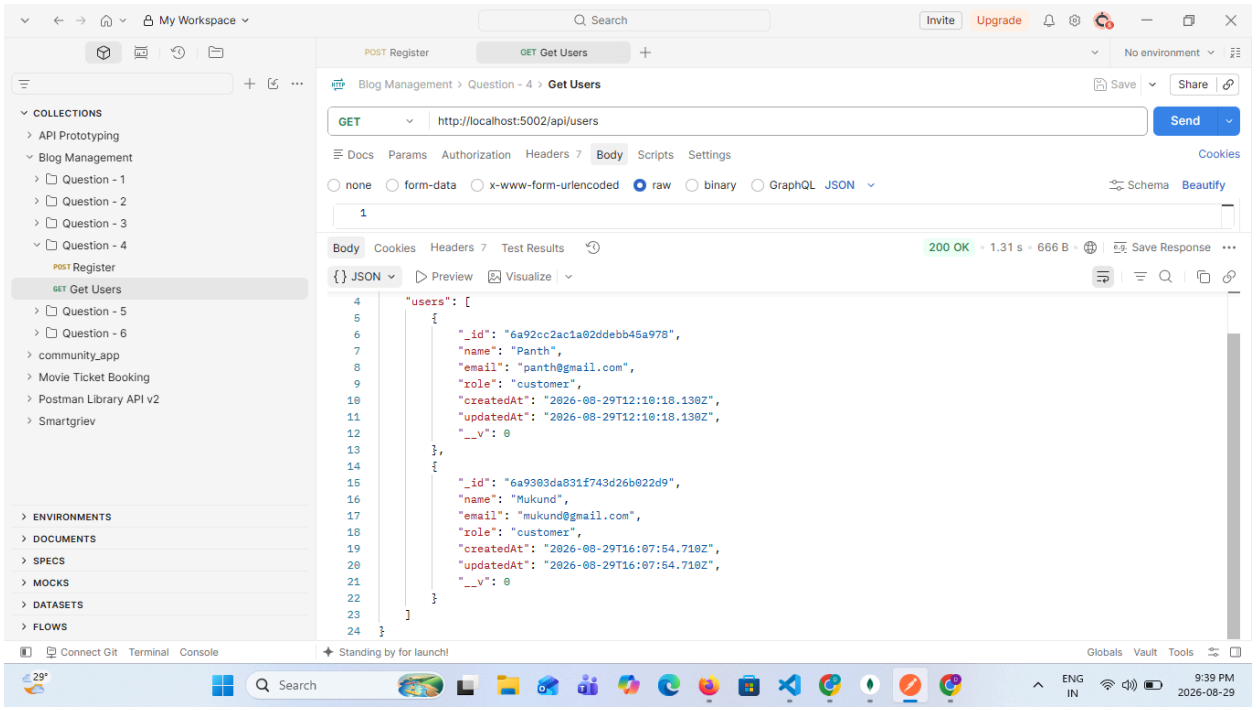
Defining Customer and Admin Roles

Scenario: The bookstore platform has two kinds of accounts — regular Customers who browse and buy books, and Admins who manage inventory, pricing, and all customer orders.

Task: Write a Mongoose User schema with a role field ('customer' / 'admin') that defaults to 'customer', so every new signup is treated as a regular customer unless explicitly promoted to admin.




My Workspace
Search
Invite
Upgrade
POST Register
No environment
Blog Management › Question - 4 › Register
Save
Share
POST
http://localhost:5002/api/auth/register
Send
Docs Params Authorization Headers 9 Body Scripts Settings
Cookies
none form-data x-www-form-urlencoded raw binary GraphQL JSON
Schema Beautify
{
"name": "Mukund",
"email": "mukund@gmail.com",
"password": "mukund@123"
}
COLLECTIONS
API Prototyping
Blog Management
Question - 1
Question - 2
Question - 3
Question - 4
POST Register
GET Get Users
Question - 5
Question - 6
community_app
Movie Ticket Booking
Postman Library API v2
Smartgriev
ENVIRONMENTS
DOCUMENTS
SPECS
MOCKS
DATASETS
FLOWS
Body Cookies Headers 7 Test Results
201 Created 209 ms 400 B
Save Response
{ } JSON Preview Visualize
{
"success": true,
"message": "User registered successfully",
"user": {
"id": "6a9303da831f743d26b022d9",
"name": "Mukund",
"email": "mukund@gmail.com",
"role": "customer"
}
}
Connect Git Terminal Console
Standing by for launch!
Globals Vault Tools
Search
ENG IN
9:38 PM
2026-08-29

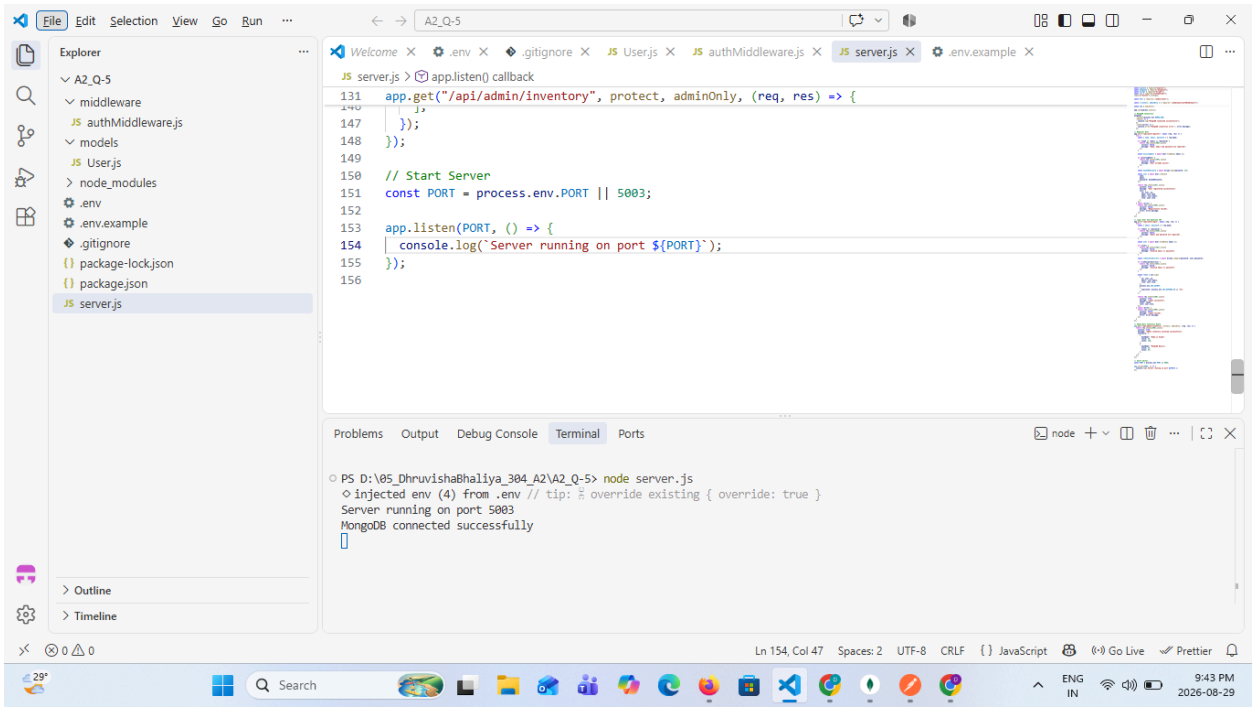

My Workspace
Search
Invite
Upgrade
POST Register
GET Get Users
No environment
Blog Management › Question - 4 › Get Users
Save
Share
GET
http://localhost:5002/api/users
Send
Docs Params Authorization Headers 7 Body Scripts Settings
Cookies
none form-data x-www-form-urlencoded raw binary GraphQL JSON
Schema Beautify
Body Cookies Headers 7 Test Results
200 OK 1.31 s 666 B
Save Response
{ } JSON Preview Visualize
"users": [
{
"_id": "6a92cc2ac1a02ddebb45a978",
"name": "Panth",
"email": "panth@gmail.com",
"role": "customer",
"createdAt": "2026-08-29T12:10:18.130Z",
"updatedAt": "2026-08-29T12:10:18.130Z",
"__v": 0
},
{
"_id": "6a9303da831f743d26b022d9",
"name": "Mukund",
"email": "mukund@gmail.com",
"role": "customer",
"createdAt": "2026-08-29T16:07:54.710Z",
"updatedAt": "2026-08-29T16:07:54.710Z",
"__v": 0
}
]
}
COLLECTIONS
API Prototyping
Blog Management
Question - 1
Question - 2
Question - 3
Question - 4
POST Register
GET Get Users
Question - 5
Question - 6
community_app
Movie Ticket Booking
Postman Library API v2
Smartgriev
ENVIRONMENTS
DOCUMENTS
SPECS
MOCKS
DATASETS
FLOWS
Connect Git Terminal Console
Standing by for launch!
Globals Vault Tools
Search
ENG IN
9:39 PM
2026-08-29

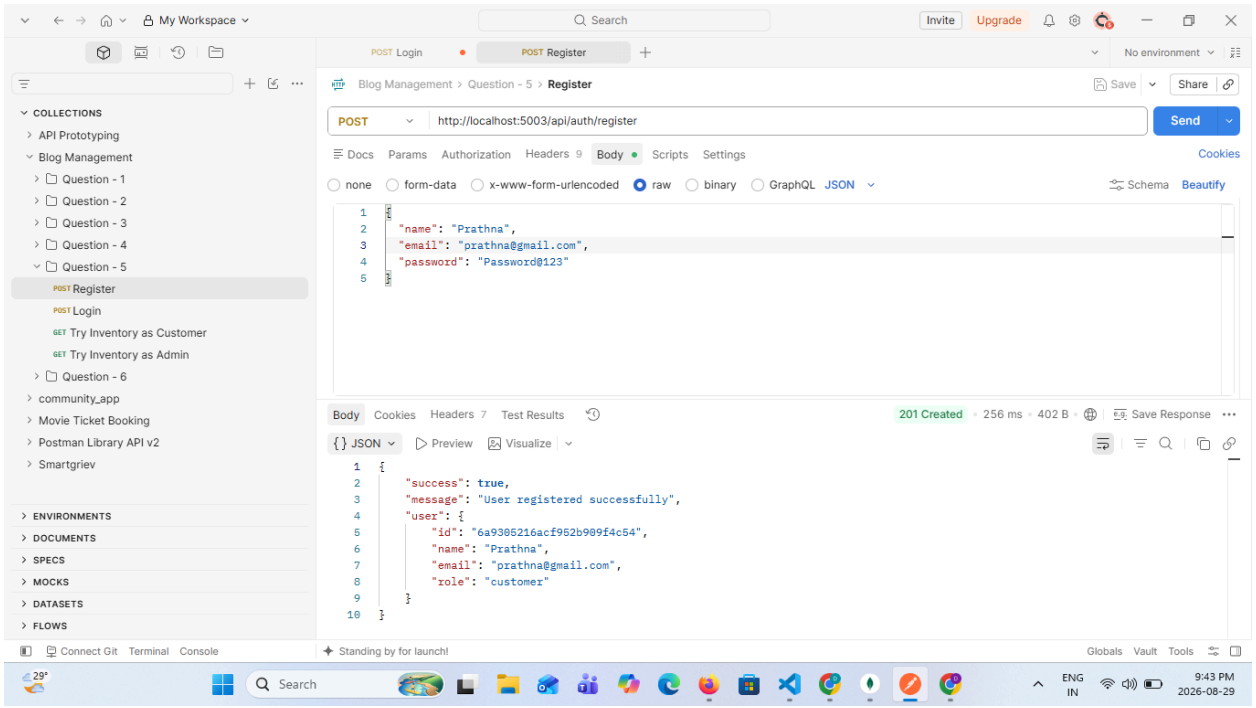
Question 5

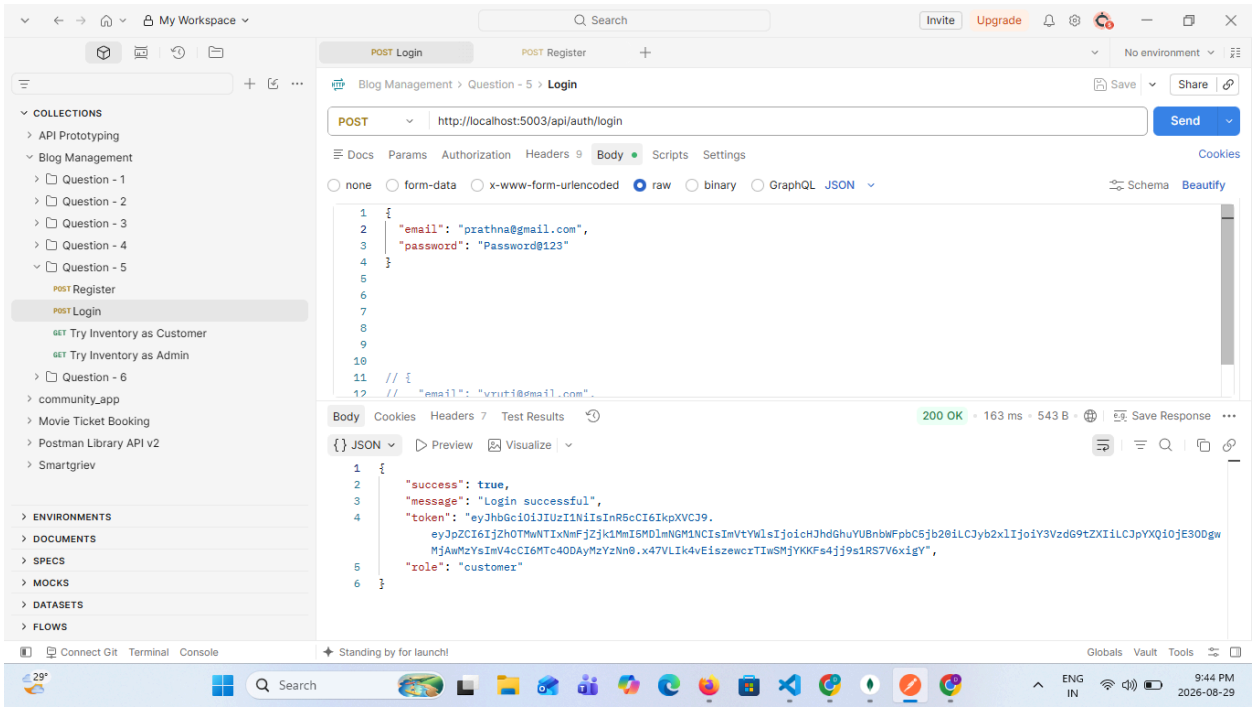
Admin-Only Access to the Inventory Dashboard

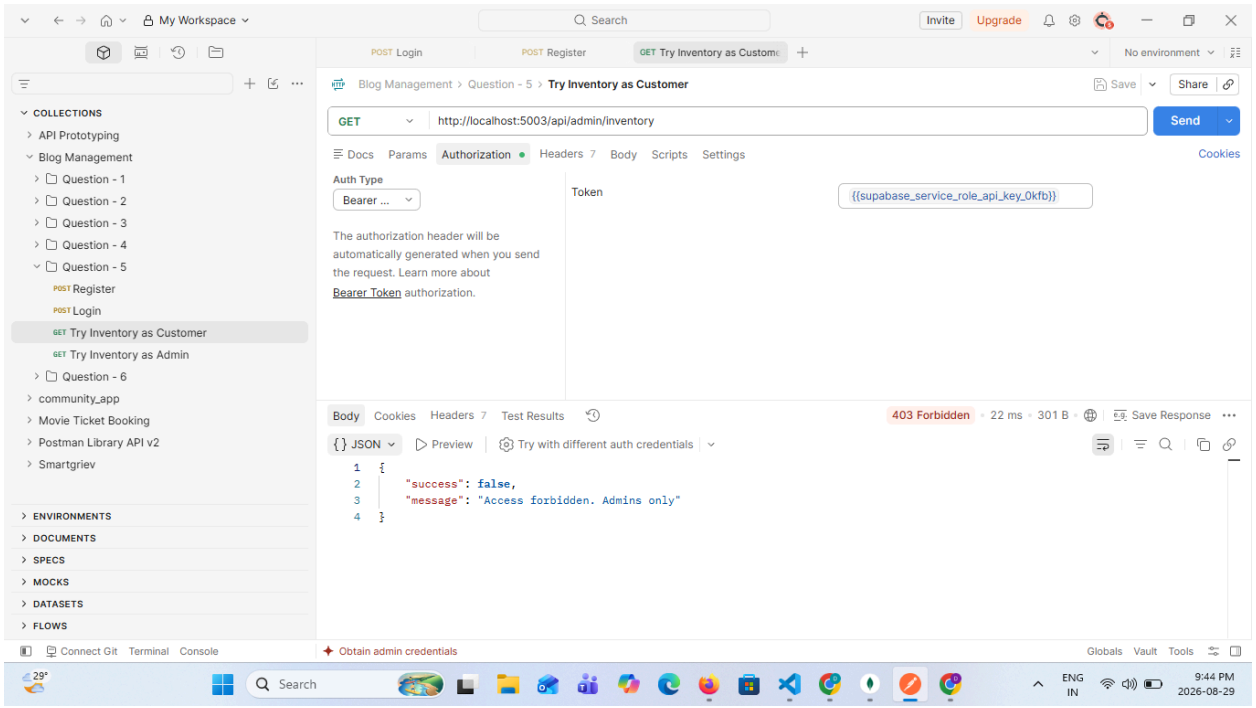
Scenario: The bookstore has an inventory management route (GET /api/admin/inventory) that lists stock levels and sales data for every book, meant only for store admins.

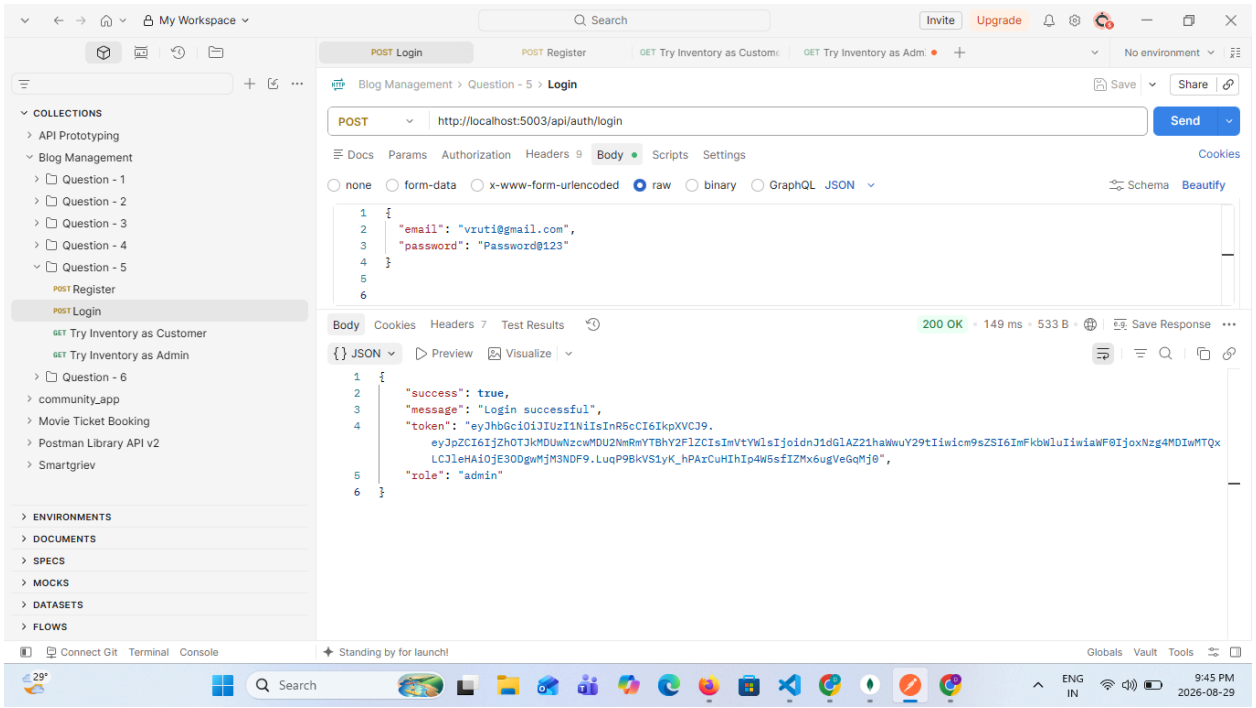
Task: Write an authorization middleware that checks the logged-in user's role and returns 403 Forbidden if the user is not an 'admin', preventing regular customers from accessing this route.

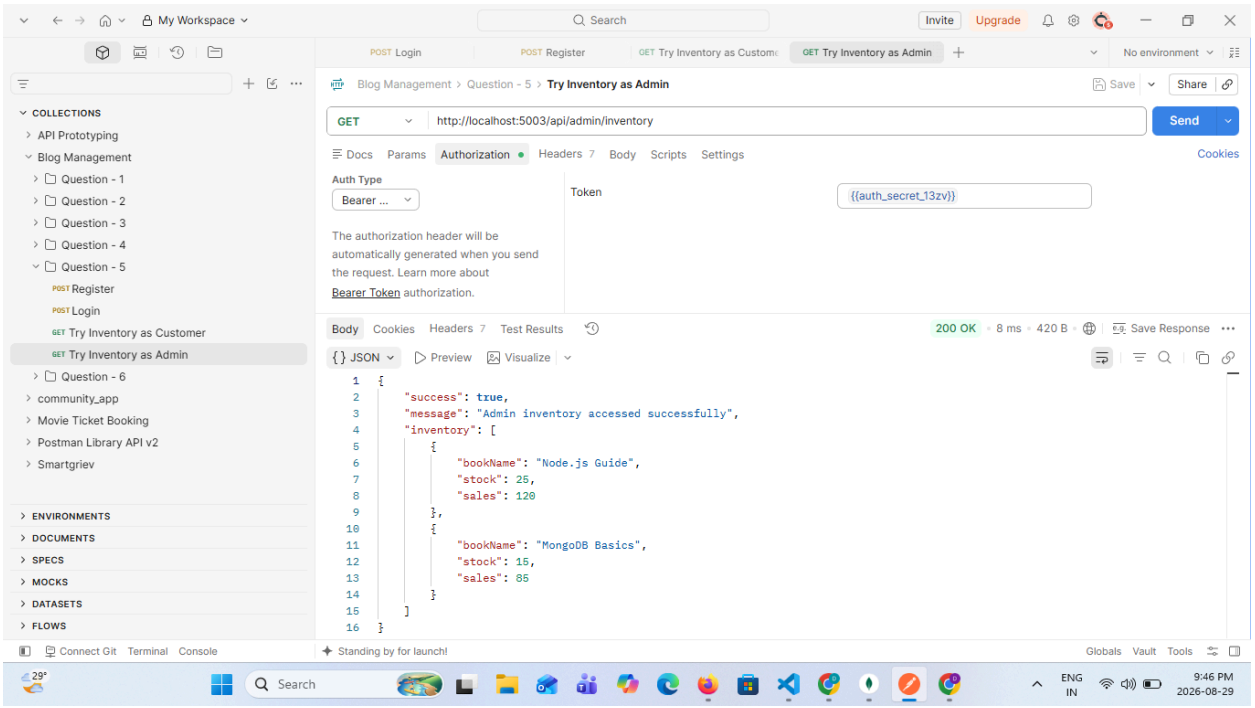



My Workspace
Search
Invite
Upgrade
POST Login
POST Register
No environment
Blog Management › Question - 5 › Register
Save
Share
POST
http://localhost:5003/api/auth/register
Send
Docs Params Authorization Headers 9 Body Scripts Settings
Cookies
none form-data x-www-form-urlencoded raw binary GraphQL JSON
Schema Beautify
{
"name": "Prathna",
"email": "prathna@gmail.com",
"password": "Password@123"
}
Body Cookies Headers 7 Test Results
201 Created 256 ms 402 B
Save Response
{ } JSON Preview Visualize
{
"success": true,
"message": "User registered successfully",
"user": {
"id": "6a9385216acf952b909f4c54",
"name": "Prathna",
"email": "prathna@gmail.com",
"role": "customer"
}
}
COLLECTIONS
API Prototyping
Blog Management
Question - 1
Question - 2
Question - 3
Question - 4
Question - 5
POST Register
POST Login
GET Try Inventory as Customer
GET Try Inventory as Admin
Question - 6
community_app
Movie Ticket Booking
Postman Library API v2
Smartgriev
ENVIRONMENTS
DOCUMENTS
SPECS
MOCKS
DATASETS
FLOWS
Connect Git Terminal Console
Standing by for launch!
Globals Vault Tools
Search
ENG IN
9:43 PM
2026-08-29


My Workspace
Search
Invite
Upgrade
POST Login
POST Register
No environment
Blog Management › Question - 5 › Login
Save
Share
POST
http://localhost:5003/api/auth/login
Send
Docs Params Authorization Headers 9 Body Scripts Settings
Cookies
none form-data x-www-form-urlencoded raw binary GraphQL JSON
Schema Beautify
{
"email": "prathna@gmail.com",
"password": "Password@123"
}
// {
// "email": "vruti@gmail.com",
Body Cookies Headers 7 Test Results
200 OK 163 ms 543 B
Save Response
{ } JSON Preview Visualize
{
"success": true,
"message": "Login successful",
"token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.
eyJpZCI6IjZhOTMwNTIxNmFjZjk1MmI5MDlmNGM1NCIsImVtYWlsIjoicHJhdGhuYUBnbWFpbC5jb20iLCJyb2xlIjoiY3VzdG9tZXIiLCJpYXQiOjE3ODgw
MjAwMzYsImV4cCI6MTc4ODAyMzYzNn0.x47VLIk4vEiszewcrTIwSMjYKKFs4jj9s1RS7V6xigY",
"role": "customer"
}
COLLECTIONS
API Prototyping
Blog Management
Question - 1
Question - 2
Question - 3
Question - 4
Question - 5
POST Register
POST Login
GET Try Inventory as Customer
GET Try Inventory as Admin
Question - 6
community_app
Movie Ticket Booking
Postman Library API v2
Smartgriev
ENVIRONMENTS
DOCUMENTS
SPECS
MOCKS
DATASETS
FLOWS
Connect Git Terminal Console
Standing by for launch!
Globals Vault Tools
Search
ENG IN
9:44 PM
2026-08-29


My Workspace
Search
Invite
Upgrade
POST Login
POST Register
GET Try Inventory as Custom
No environment
Blog Management > Question - 5 > Try Inventory as Customer
Save
Share
GET
http://localhost:5003/api/admin/inventory
Send
Docs
Params
Authorization
Headers 7
Body
Scripts
Settings
Cookies
Auth Type
Bearer ...
The authorization header will be automatically generated when you send the request. Learn more about Bearer Token authorization.
Token
{{supabase_service_role_api_key_0kfb}}
Body
Cookies
Headers 7
Test Results
403 Forbidden
22 ms
301 B
Save Response
JSON
Preview
Try with different auth credentials
{
"success": false,
"message": "Access forbidden. Admins only"
}
COLLECTIONS
API Prototyping
Blog Management
Question - 1
Question - 2
Question - 3
Question - 4
Question - 5
POST Register
POST Login
GET Try Inventory as Customer
GET Try Inventory as Admin
Question - 6
community_app
Movie Ticket Booking
Postman Library API v2
Smartgriev
ENVIRONMENTS
DOCUMENTS
SPECS
MOCKS
DATASETS
FLOWS
Connect Git
Terminal
Console
Obtain admin credentials
Globals
Vault
Tools
Search
ENG
IN
9:44 PM
2026-08-29


My Workspace
Search
Invite
Upgrade
POST Login
POST Register
GET Try Inventory as Custom
GET Try Inventory as Adm
No environment
Blog Management > Question - 5 > Login
Save
Share
POST
http://localhost:5003/api/auth/login
Send
Docs
Params
Authorization
Headers 9
Body
Scripts
Settings
Cookies
none
form-data
x-www-form-urlencoded
raw
binary
GraphQL
JSON
Schema
Beautify
{
"email": "vruti@gmail.com",
"password": "Password@123"
}
Body
Cookies
Headers 7
Test Results
200 OK
149 ms
533 B
Save Response
JSON
Preview
Visualize
{
"success": true,
"message": "Login successful",
"token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.
eyJpZCI6IjZhOTJkMDUwNzcwMDU2NmRmYTBhY2FlZCIsImVtYWlsIjoidnJ1dGlAZ21haWwuY29tIiwicm9sZSI6ImFkbWluIiwiaWF0IjoxNzg4MDIwMTQx
LCJleHAiOjE3ODgwMjM3NDF9.LuqP9BkVS1yK_hPArCuHIhIp4W5sfIZMx6ugVeGqMj0",
"role": "admin"
}
COLLECTIONS
API Prototyping
Blog Management
Question - 1
Question - 2
Question - 3
Question - 4
Question - 5
POST Register
POST Login
GET Try Inventory as Customer
GET Try Inventory as Admin
Question - 6
community_app
Movie Ticket Booking
Postman Library API v2
Smartgriev
ENVIRONMENTS
DOCUMENTS
SPECS
MOCKS
DATASETS
FLOWS
Connect Git
Terminal
Console
Standing by for launch!
Globals
Vault
Tools
Search
ENG
IN
9:45 PM
2026-08-29

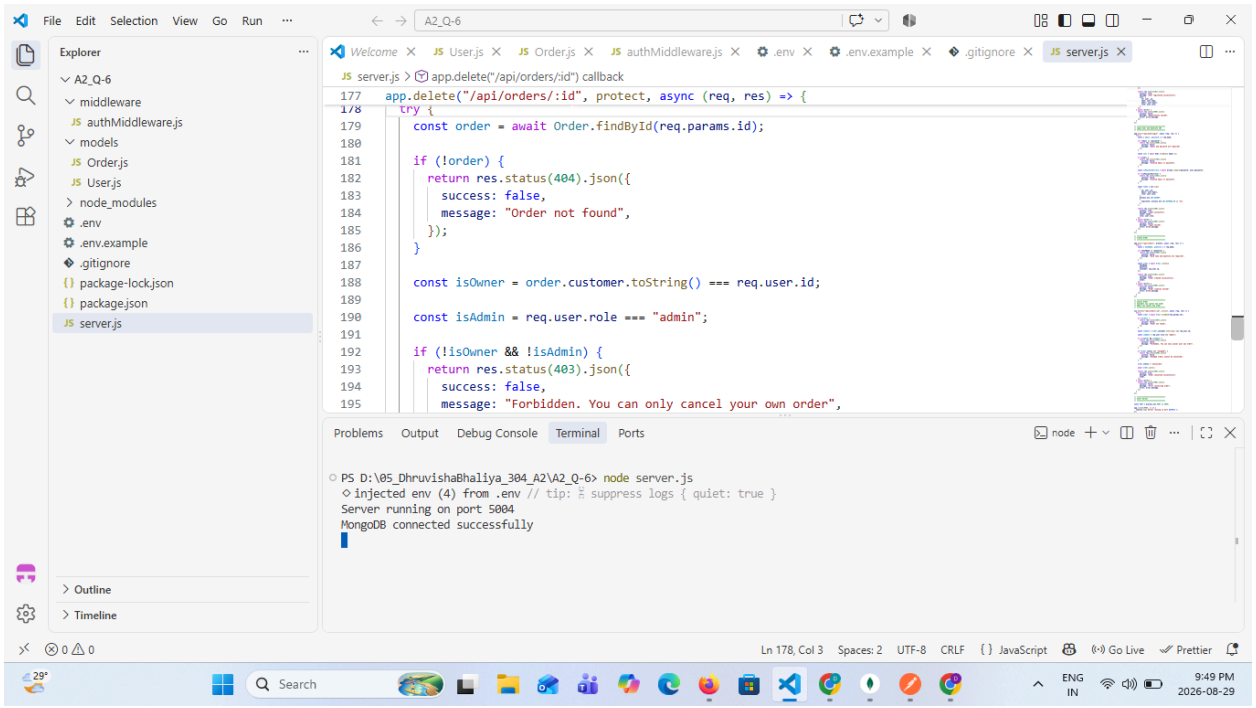


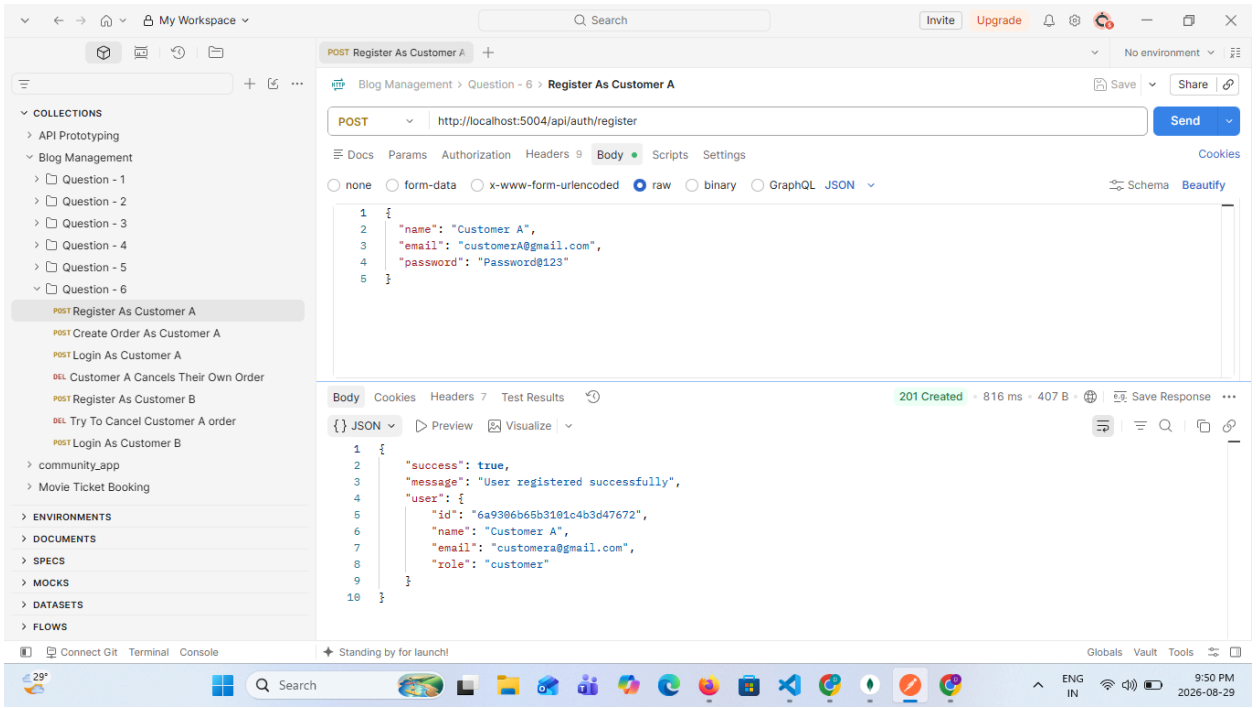
Question 6

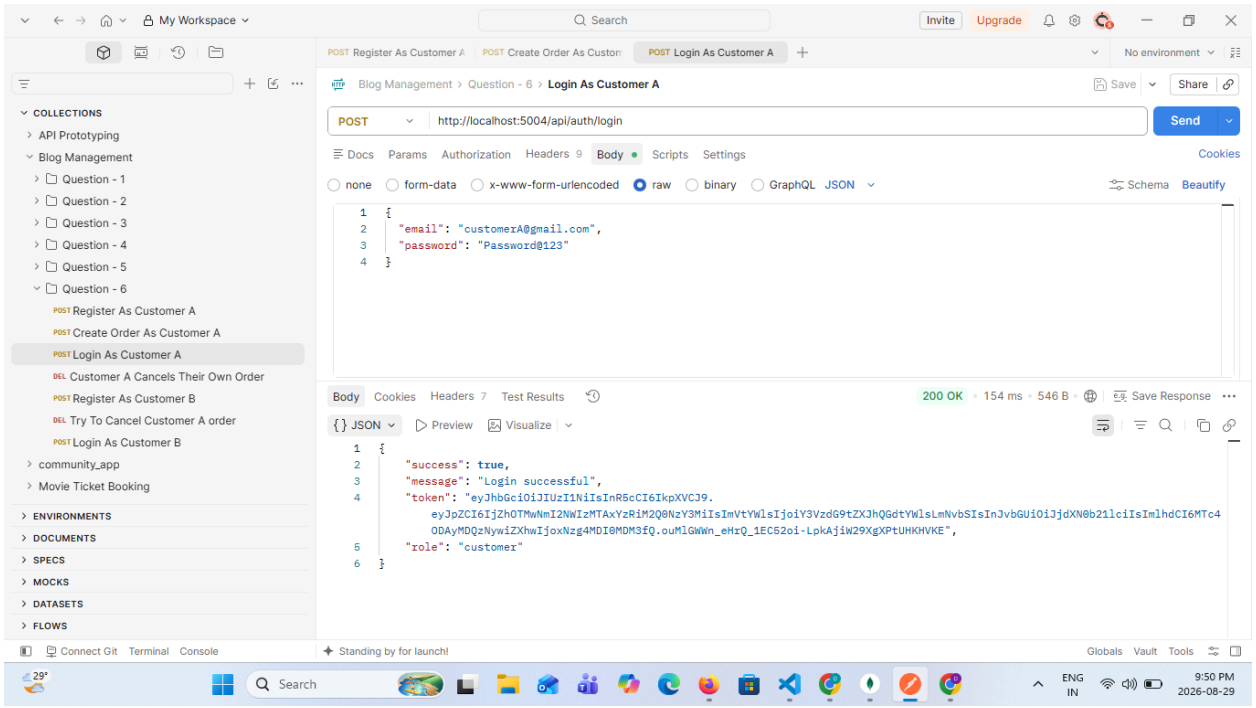
Cancelling an Order — Customer or Admin Only

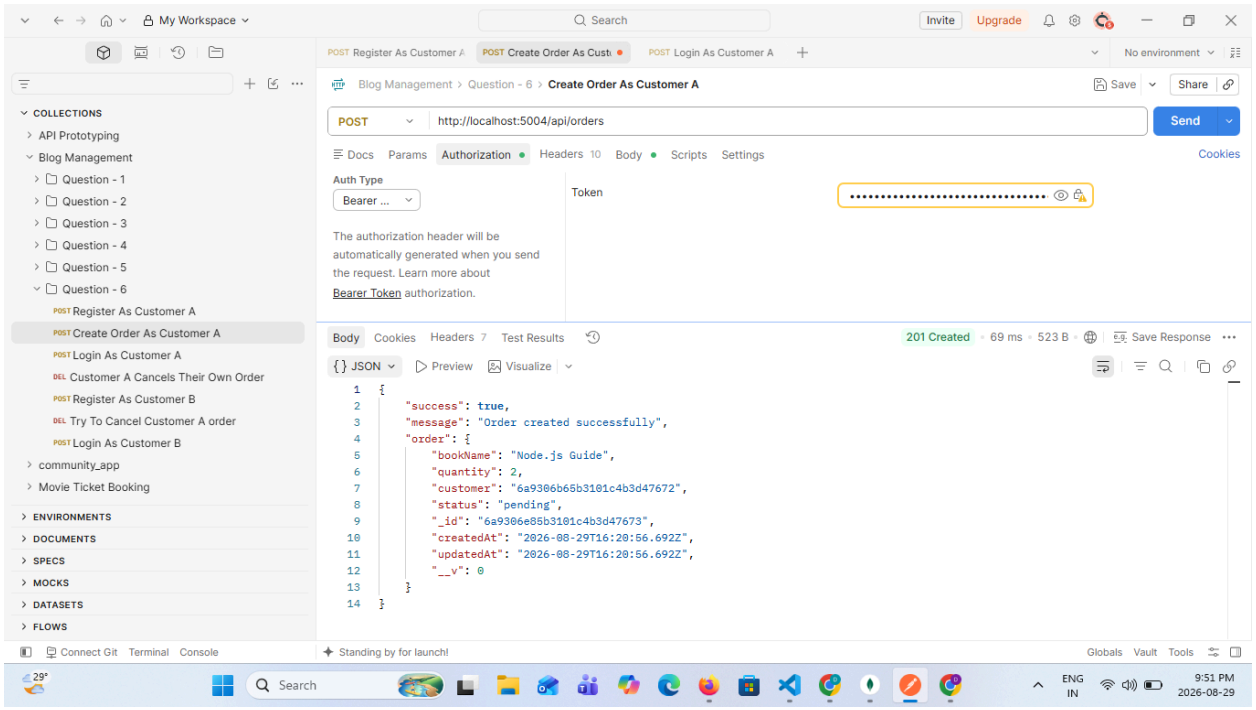
Scenario: A customer should be able to cancel their own order before it ships. Separately, an admin should be able to cancel any customer's order — for example, when a book goes out of stock.

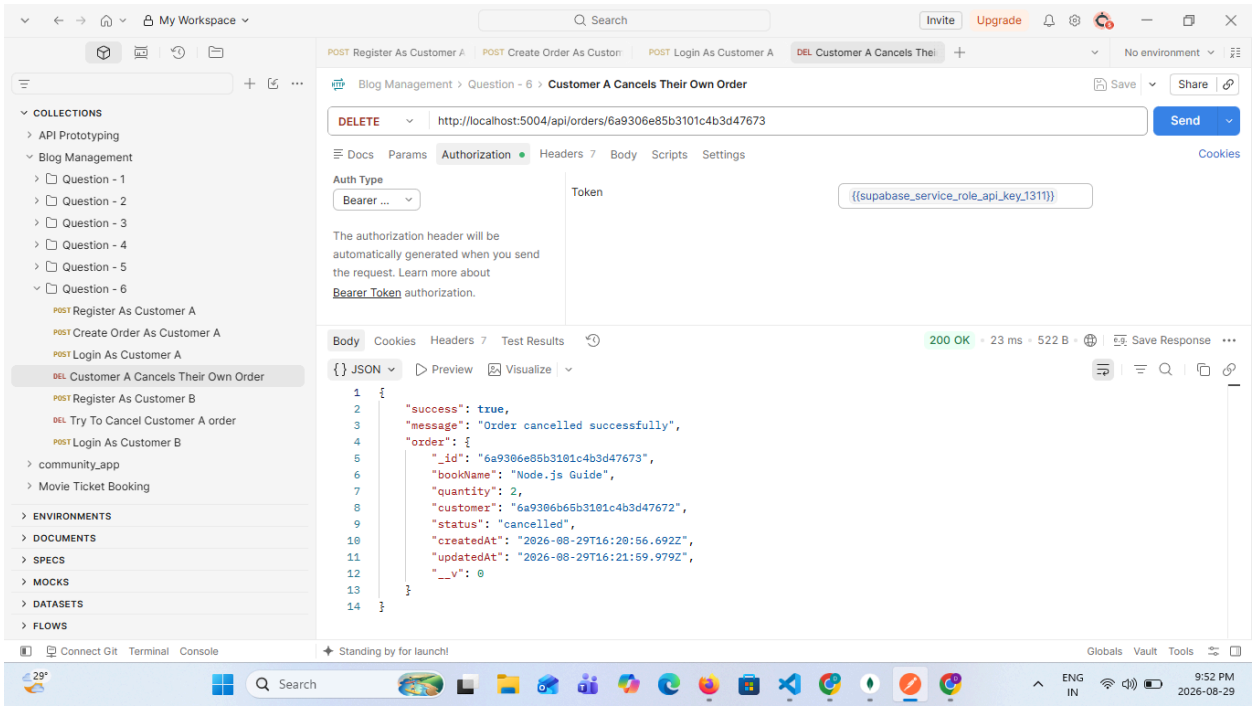
Task: Write a DELETE /api/orders/:id handler that allows an order to be cancelled only if the logged-in user is the customer who placed the order, or holds the admin role, and returns 403 Forbidden otherwise.

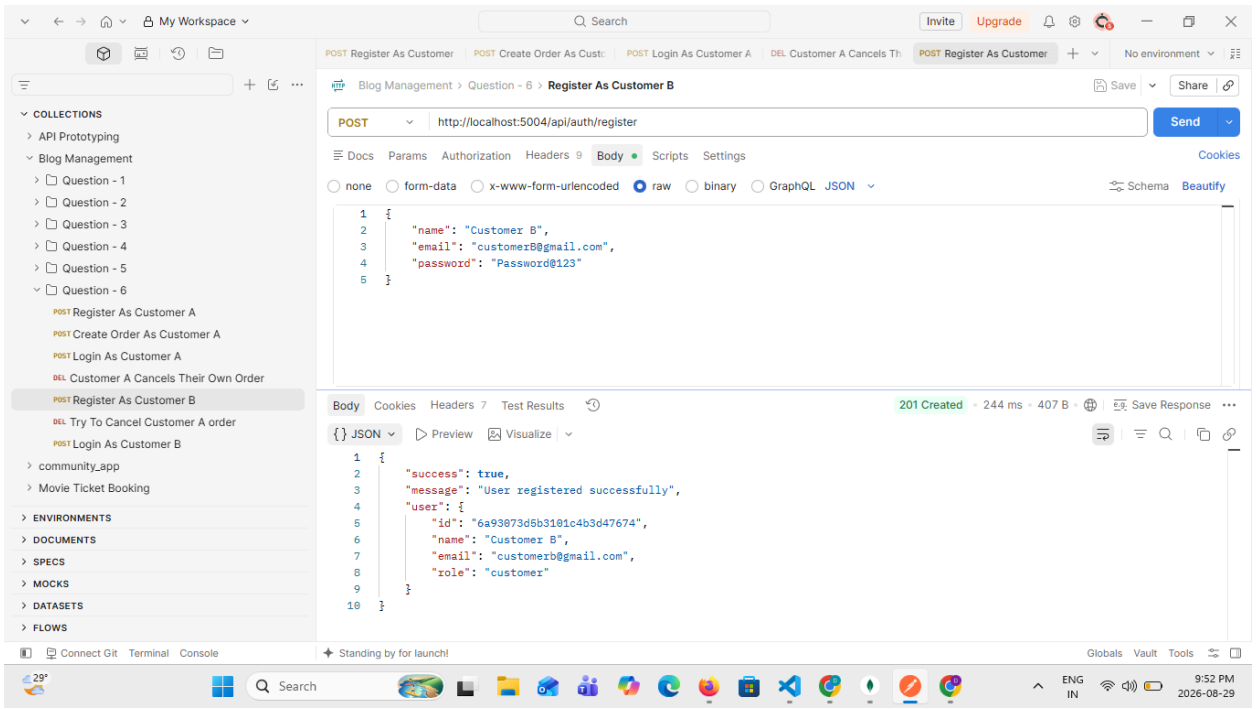

File Edit Selection View Go Run
A2_Q-6
Explorer
A2_Q-6
middleware
authMiddleware.js
models
Order.js
User.js
node_modules
.env
.env.example
.gitignore
package-lock.json
package.json
server.js
Outline
Timeline
Welcome
User.js
Order.js
authMiddleware.js
.env
.env.example
.gitignore
server.js
server.js > app.delete("/api/orders/:id") callback
177 app.delete("/api/orders/:id", protect, async (req, res) => {
178 try {
179 const order = await Order.findById(req.params.id);
180
181 if (!order) {
182 return res.status(404).json({
183 success: false,
184 message: "Order not found",
185 });
186 }
187
188 const isOwner = order.customer.toString() === req.user.id;
189
190 const isAdmin = req.user.role === "admin";
191
192 if (!isOwner && !isAdmin) {
193 return res.status(403).json({
194 success: false,
195 message: "Forbidden. You can only cancel your own order",
Problems Output Debug Console Terminal Ports
node
PS D:\05_DhruvishaBhaliya_304_A2\A2_Q-6> node server.js
injected env (4) from .env // tip: suppress logs { quiet: true }
Server running on port 5004
MongoDB connected successfully
Ln 178, Col 3 Spaces: 2 UTF-8 CRLF JavaScript Go Live Prettier
Search
ENG IN
9:49 PM
2026-08-29

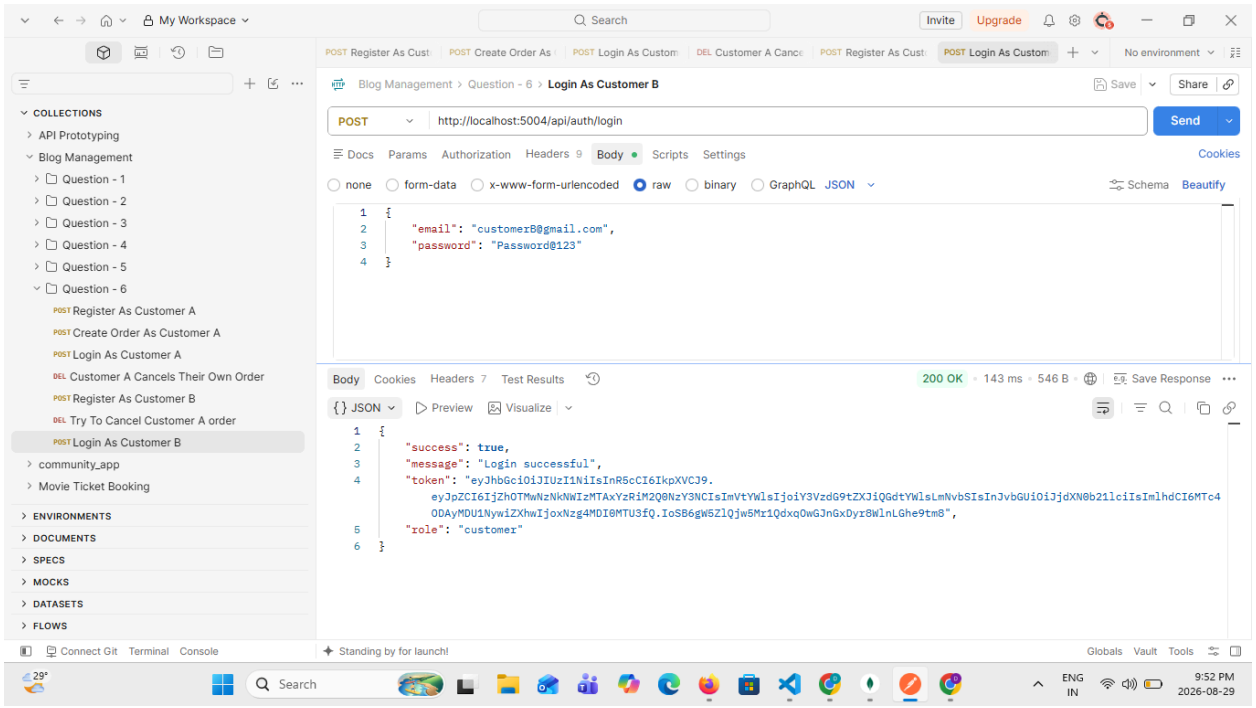

My Workspace
Search
Invite
Upgrade
POST Register As Customer A
No environment
Blog Management > Question - 6 > Register As Customer A
Save
Share
COLLECTIONS
API Prototyping
Blog Management
Question - 1
Question - 2
Question - 3
Question - 4
Question - 5
Question - 6
POST Register As Customer A
POST Create Order As Customer A
POST Login As Customer A
DEL Customer A Cancels Their Own Order
POST Register As Customer B
DEL Try To Cancel Customer A order
POST Login As Customer B
community_app
Movie Ticket Booking
ENVIRONMENTS
DOCUMENTS
SPECS
MOCKS
DATASETS
FLOWS
POST
http://localhost:5004/api/auth/register
Send
Docs Params Authorization Headers 9 Body Scripts Settings
Cookies
none form-data x-www-form-urlencoded raw binary GraphQL JSON
Schema Beautify
1 {
2 "name": "Customer A",
3 "email": "customerA@gmail.com",
4 "password": "Password@123"
5 }
Body Cookies Headers 7 Test Results
201 Created 816 ms 407 B
Save Response
{ } JSON Preview Visualize
1 {
2 "success": true,
3 "message": "User registered successfully",
4 "user": {
5 "id": "6a9306b65b3101c4b3d47672",
6 "name": "Customer A",
7 "email": "customera@gmail.com",
8 "role": "customer"
9 }
10 }
Connect Git Terminal Console
Standing by for launch!
Globals Vault Tools
Search
ENG IN
9:50 PM
2026-08-29

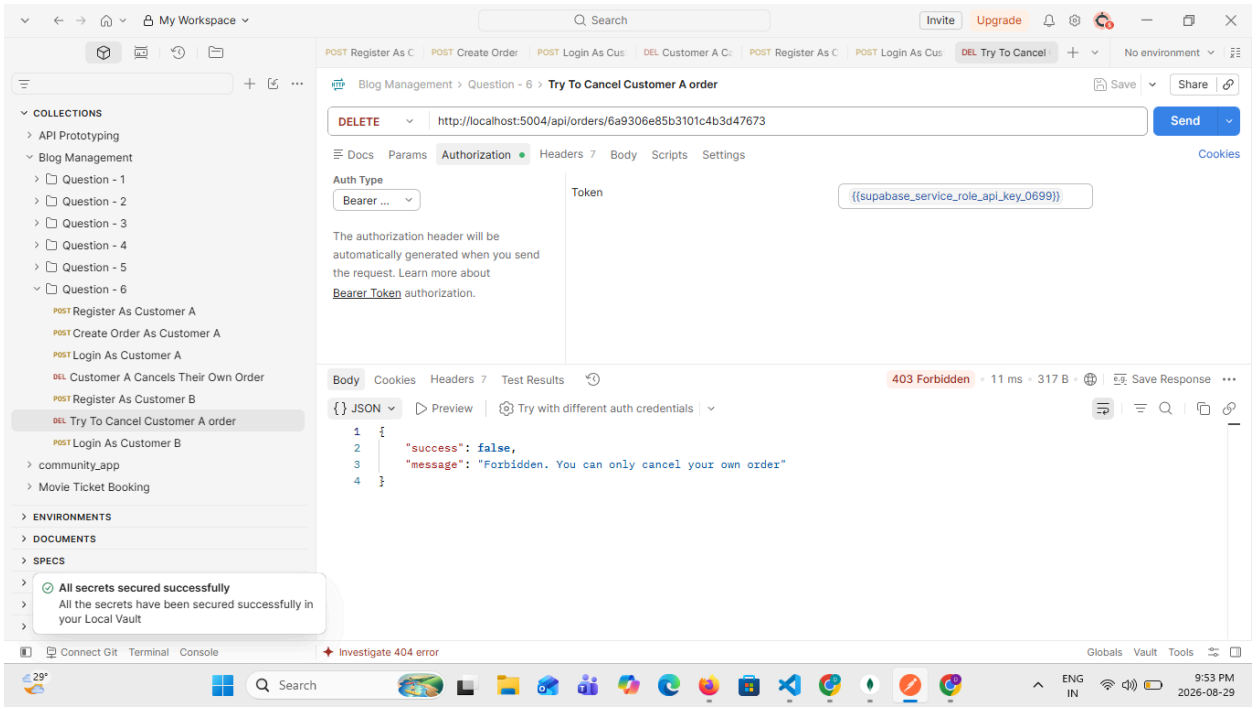

My Workspace
Search
Invite
Upgrade
POST Register As Customer A
POST Create Order As Custon
POST Login As Customer A
No environment
Blog Management › Question - 6 › Login As Customer A
Save
Share
COLLECTIONS
API Prototyping
Blog Management
Question - 1
Question - 2
Question - 3
Question - 4
Question - 5
Question - 6
POST Register As Customer A
POST Create Order As Customer A
POST Login As Customer A
DEL Customer A Cancels Their Own Order
POST Register As Customer B
DEL Try To Cancel Customer A order
POST Login As Customer B
community_app
Movie Ticket Booking
ENVIRONMENTS
DOCUMENTS
SPECS
MOCKS
DATASETS
FLOWS
POST
http://localhost:5004/api/auth/login
Send
Docs
Params
Authorization
Headers 9
Body
Scripts
Settings
Cookies
none
form-data
x-www-form-urlencoded
raw
binary
GraphQL
JSON
Schema
Beautify
{
"email": "customerA@gmail.com",
"password": "Password@123"
}
Body
Cookies
Headers 7
Test Results
200 OK
154 ms
546 B
Save Response
{ } JSON
Preview
Visualize
{
"success": true,
"message": "Login successful",
"token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.
eyJpZCI6IjZhOTMwNmI2NWIzMTAxYzRiM2Q0NzY3MiIsImVtYWlsIjoiY3VzdG9tZXJhQGdtYWlsLmNvbSIsInJvbGUiOiJjdXN0b21lciIsImlhdCI6MTc4ODAyMDQzNywiZXhwIjoxNzg4MDI0MDM3fQ.ouMlGWWn_eHzQ_1EC52oi-LpkAjiW29XgXPtUHKHVKE",
"role": "customer"
}
Connect Git
Terminal
Console
Standing by for launch!
Globals
Vault
Tools
Search
ENG
IN
9:50 PM
2026-08-29


My Workspace
Search
Invite
Upgrade
POST Register As Customer A
POST Create Order As Custo
POST Login As Customer A
No environment
Blog Management › Question - 6 › Create Order As Customer A
Save
Share
COLLECTIONS
API Prototyping
Blog Management
Question - 1
Question - 2
Question - 3
Question - 4
Question - 5
Question - 6
POST Register As Customer A
POST Create Order As Customer A
POST Login As Customer A
DEL Customer A Cancels Their Own Order
POST Register As Customer B
DEL Try To Cancel Customer A order
POST Login As Customer B
community_app
Movie Ticket Booking
ENVIRONMENTS
DOCUMENTS
SPECS
MOCKS
DATASETS
FLOWS
POST
http://localhost:5004/api/orders
Send
Docs
Params
Authorization
Headers 10
Body
Scripts
Settings
Cookies
Auth Type
Bearer ...
Token
The authorization header will be automatically generated when you send the request. Learn more about Bearer Token authorization.
Body
Cookies
Headers 7
Test Results
201 Created
69 ms
523 B
Save Response
{ } JSON
Preview
Visualize
{
"success": true,
"message": "Order created successfully",
"order": {
"bookName": "Node.js Guide",
"quantity": 2,
"customer": "6a9306b65b3101c4b3d47672",
"status": "pending",
"_id": "6a9306e85b3101c4b3d47673",
"createdAt": "2026-08-29T16:20:56.692Z",
"updatedAt": "2026-08-29T16:20:56.692Z",
"__v": 0
}
}
Connect Git
Terminal
Console
Standing by for launch!
Globals
Vault
Tools
Search
ENG
IN
9:51 PM
2026-08-29


Blog Management › Question - 6 › Customer A Cancels Their Own Order
DELETE
http://localhost:5004/api/orders/6a9306e85b3101c4b3d47673
Send
Auth Type
Bearer ...
The authorization header will be automatically generated when you send the request. Learn more about Bearer Token authorization.
Token
{{supabase_service_role_api_key_1311}}
200 OK
23 ms
522 B
{
"success": true,
"message": "Order cancelled successfully",
"order": {
"_id": "6a9306e85b3101c4b3d47673",
"bookName": "Node.js Guide",
"quantity": 2,
"customer": "6a9306b65b3101c4b3d47672",
"status": "cancelled",
"createdAt": "2026-08-29T16:20:56.692Z",
"updatedAt": "2026-08-29T16:21:59.979Z",
"__v": 0
}
}


Blog Management › Question - 6 › Register As Customer B
POST
http://localhost:5004/api/auth/register
Send
{
"name": "Customer B",
"email": "customerB@gmail.com",
"password": "Password@123"
}
201 Created
244 ms
407 B
{
"success": true,
"message": "User registered successfully",
"user": {
"id": "6a93073d5b3101c4b3d47674",
"name": "Customer B",
"email": "customerb@gmail.com",
"role": "customer"
}
}


Blog Management › Question - 6 › Login As Customer B
POST
http://localhost:5004/api/auth/login
Send
{
"email": "customerB@gmail.com",
"password": "Password@123"
}
200 OK
{
"success": true,
"message": "Login successful",
"token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.
eyJpZCI6IjZhOTMwNzNkNWIzMTAxYzRiM2Q0NzY3NCIsImVtYWlsIjoiY3VzdG9tZXJiQGdtYWlsLmNvbSIsInJvbGUiOiJjdXN0b21lciIsImlhdCI6MTc4ODAyMDU1NywiZXhwIjoxNzg4MDI0MTU3fQ.IoSB6gW5ZlQjw5Mr1QdxqOwGJnGxDyr8WlnLGhe9tm8",
"role": "customer"
}


Blog Management › Question - 6 › Try To Cancel Customer A order
DELETE
http://localhost:5004/api/orders/6a9306e85b3101c4b3d47673
Send
Auth Type
Bearer ...
Token
{{supabase_service_role_api_key_0699}}
403 Forbidden
{
"success": false,
"message": "Forbidden. You can only cancel your own order"
}
All secrets secured successfully
All the secrets have been secured successfully in your Local Vault